Practice Exam 1

SDU · Internet of Things · Lectures 1–12

QUESTION	MODEL ANSWER

Section 1 — Multiple Choice

QUESTION	MODEL ANSWER
MC1 The Nyquist-Shannon sampling theorem states that to accurately reconstruct a signal you must sample at a rate that is: • A) Equal to the highest frequency component in the signal • B) At least twice the highest frequency component in the signal • C) At least three times the highest frequency component in the signal • D) At most half the highest frequency component in the signal	**B** Nyquist: sample at > 2× the highest frequency. Sampling at exactly 2× is the theoretical minimum; in practice you need a bit more.
MC2 In Elixir's GenServer, which callback *returns a reply* to the calling process? • A) `handle_cast/2` • B) `handle_info/2` • C) `handle_call/3` • D) `handle_continue/2`	**C** `handle_call/3` returns `{:reply, value, new_state}`. The other callbacks return `{:noreply, ...}` and send no reply.
MC3 Which MQTT QoS level guarantees that a message is delivered *exactly once*? • A) QoS 0 — at most once • B) QoS 1 — at least once • C) QoS 2 — exactly once • D) QoS 3 — ordered delivery	**C** QoS 2 uses a four-step handshake (PUBLISH → PUBREC → PUBREL → PUBCOMP). QoS 1 is *at least* once (duplicates possible).
MC4 A C `union` differs from a C `struct` in that: • A) A union can only hold integer types • B) All members of a union share the same memory location • C) A union cannot be nested inside a struct • D) A union always allocates memory for all its members simultaneously	**B** All union members are laid out starting at the same memory address. Size = size of the largest member. Only one member is valid at a time.
MC5 In the IoT three-tier model, which tier is most appropriate for hard real-time control — e.g., detecting a threshold breach and triggering an actuator within milliseconds?	**C**

Edge processing is local, has no network dependency, and meets sub-millisecond latency requirements. Cloud round-trips add 10–100+ ms and fail when connectivity is lost.

- A) Cloud
- B) Fog
- C) Edge
- D) All tiers are equally suitable

MC6

C

Which LoRa spreading factor gives the *longest range* but the *lowest data rate*?

SF12 gives the longest range (~15 km line-of-sight) and slowest data rate (~250 bps). SF7 is fastest and shortest-range. There is no SF15.

- A) SF7
- B) SF9
- C) SF12
- D) SF15

MC7

B

In Elixir, the `=` operator is:

In Elixir `=` is the match operator. `x = 5` succeeds because `x` is unbound; `5 = x` also succeeds (x is already 5); `6 = x` raises a MatchError.

- A) An assignment operator
- B) A match operator
- C) A structural equality test
- D) A binding operator that always creates a new variable

MC8

C

What is the primary function of the Low-Power Listening (LPL) layer in the TinyOS network stack?

LPL periodically wakes the radio for a brief listen, then sleeps again. Senders prepend a long preamble so the receiver catches it. Dramatically reduces idle listening energy.

- A) Encrypting packets before transmission
- B) Routing packets across multiple hops
- C) Duty-cycling the radio to save energy
- D) Retransmitting lost packets

MC9

B

In RDF, a triple is composed of:

Every RDF statement is a triple: (subject, predicate, object). All three are identified by URIs (or literals for objects).

- A) Class, property, value
- B) Subject, predicate, object
- C) Entity, attribute, relationship
- D) Node, edge, label

MC10

C

The `atten` field in an ESP32 ADC channel configuration controls:

`ADC_ATTEN_DB_11` sets ~11 dB attenuation, widening the measurable range to ~0–3.9 V. Lower attenuation gives better resolution over a smaller voltage range.

- A) The sampling frequency
- B) The bit resolution of the conversion

• C) The input voltage attenuation, which determines the measurable voltage range • D) The number of samples to average before returning a reading	
MC11 Which statement about LoRaWAN duty cycle is correct? • A) Duty cycle limits are optional and only enforced by network operators • B) Duty cycle is a regulatory constraint limiting what fraction of time a device may transmit • C) Duty cycle only applies to LoRaWAN Class C devices • D) Higher spreading factors are exempt from duty cycle limits	**B** In the EU (and most regions) ISM-band devices must not exceed a duty cycle fraction (e.g., 1% on 868 MHz sub-bands). This is a legal requirement, not just a recommendation.
MC12 In a FreeRTOS project, what is the purpose of a binary semaphore? • A) Transferring large blocks of data between tasks • B) Signalling that an event has occurred or synchronising access to a shared resource • C) Replacing the need for task priorities • D) Allocating heap memory safely across tasks	**B** A binary semaphore signals between tasks (e.g., ISR signals a task that data is ready) or protects a critical section. It is *not* for data transfer — use a queue for that.

Section 2 — Multiple Answer

QUESTION	MODEL ANSWER
MA1 Which of the following are valid MQTT QoS levels? • A) QoS 0 — fire and forget • B) QoS 1 — at least once • C) QoS 2 — exactly once • D) QoS 3 — guaranteed ordering • E) QoS 4 — acknowledged delivery	**MA1** A, B, C** QoS 0, 1, and 2 are the only defined levels in the MQTT specification. QoS 3 and 4 do not exist.
MA2 Which properties make C suitable for use at the Edge tier? Select all that apply. • A) No garbage collector — timing is deterministic • B) Compiles to bare-metal machine code with minimal runtime footprint • C) Direct hardware access via pointer arithmetic and bit manipulation • D) Strong memory-safety guarantees prevent buffer overflows • E) Low-level control without unnecessary runtime overhead	**MA2** A, B, C, E** D is wrong — C has *no* memory-safety guarantees; buffer overflows are a well-known C vulnerability. The other four are correct and are the primary reasons C dominates embedded development.
MA3 Under what conditions does GDPR apply to occupancy sensor data?	**MA3** A, B, D**

C is wrong — data does not need to contain a name to be personal data; indirect identification suffices. E is wrong — GDPR applies when identification *is* realistically possible, not unconditionally. A, B, and D all describe situations where the data can identify a specific natural person.

- A) When the data can be used — alone or in combination — to identify a natural person
- B) When desk assignments link occupancy records to named employees
- C) Only when names are explicitly stored alongside the sensor readings
- D) When the building has exactly one person per monitored area, making records uniquely attributable
- E) Always, regardless of whether identification is realistically possible

MA4

Which of the following are Elixir/OTP supervision strategies?

MA4 A, B, C, E**

D (`:all_for_one`) does not exist in OTP — `:one_for_all` is the closest real option. All four real strategies (`:one_for_one`, `:one_for_all`, `:rest_for_one`, `:simple_one_for_one`) are correct.

- A) `:one_for_one` — restart only the failed child
- B) `:one_for_all` — restart all children when one fails
- C) `:rest_for_one` — restart the failed child and all children started after it
- D) `:all_for_one` — restart all children regardless of which one failed
- E) `:simple_one_for_one` — for dynamically spawned homogeneous workers

MA5

Which are consequences of sampling a signal *below* the Nyquist rate?

MA5 A, B, D**

C is wrong — ADC hardware has no concept of Nyquist; it always returns a number. E is wrong — anti-aliasing filters are analogue hardware placed *before* the ADC; they don't correct sub-Nyquist sampling in software. A, B, and D all describe aliasing and its direct consequences.

- A) Aliasing — high-frequency components appear as artefacts at lower frequencies
- B) The original signal cannot be perfectly reconstructed from the samples
- C) The ADC hardware returns NaN or overflow values
- D) Frequencies above the Nyquist limit are mirrored (folded) into the sampled spectrum
- E) The anti-aliasing filter automatically corrects the problem in software

MA6

Which statements about LoRa and LoRaWAN are correct?

MA6 A, B, D**

C is wrong — LoRa operates in *unlicensed* ISM bands (868 MHz EU, 915 MHz US). E is wrong — Class A devices open two short receive windows only *after* a transmission; the gateway cannot send downlink at will. Class C devices listen continuously.

- A) LoRa is a physical-layer chirp spread-spectrum modulation technique
- B) LoRaWAN adds a MAC layer, network server, and application server on top of LoRa
- C) LoRa operates exclusively in licensed frequency bands

• D) Higher spreading factors give longer range at the cost of lower data rate and longer airtime • E) LoRaWAN Class A devices can receive downlink messages at any time the gateway chooses	
MA7 Which of the following are things that go into a C header file? • A) Function declarations (prototypes) • B) Type definitions (`typedef`, `struct`, `enum`, `union`) • C) The full implementation body of functions • D) Macro definitions (`#define` constants) • E) Include guards (`#ifndef` / `#define` / `#endif`)	**MA7** A, B, D, E** C is wrong — function implementations belong in `.c` files. Putting them in headers causes multiple-definition linker errors. A, B, D, and E are all correct.

Section 3 — Short Answer

QUESTION	MODEL ANSWER
SA1 What is a C `union`, and how does it differ from a `struct`?	A `union` is a C data type where all declared members occupy the *same* memory address. Its total size equals the size of its largest member. At any given moment only one member holds a valid value — writing to one member overwrites the others. A `struct` allocates separate, contiguous memory for every member, so all members can be valid simultaneously. Unions are used when a variable needs to represent one of several mutually exclusive types (e.g., a sensor reading that is either an integer count or a float voltage).
SA2 A FreeRTOS task function has this signature: `void task_name(void *pvParameters)`. Why does it return `void`, and what is the purpose of the `void *` parameter?	The function returns `void` because FreeRTOS tasks are designed to run in an infinite loop and never return. If a task finishes, it must call `vTaskDelete(NULL)` to remove itself from the scheduler — returning from the function would leave the CPU with no valid return address and crash the system. The `void *` parameter is a generic pointer. FreeRTOS's scheduler calls all tasks through the same uniform function pointer type. The `void *` allows the creator to pass any configuration struct or value at task creation time without changing the scheduler's interface.
SA3 To add a new `.c`/`.h` file pair to an ESP-IDF / FreeRTOS project, what must you change and why?	You must add the `.c` file to the `SRCS` list in the component's `CMakeLists.txt`, and ensure the directory containing the `.h` file is listed in `INCLUDE_DIRS` (also in `CMakeLists.txt`). The ESP-IDF build system uses CMake: only files listed in `SRCS` are compiled into the object archive, and only directories in `INCLUDE_DIRS` are searched when resolving `#include` directives. Without this, the linker cannot find the compiled object file and the compiler cannot find the header.
SA4	With `cleanSession=false`, the broker retains the client's subscriptions and queues any QoS 1 or QoS 2 messages that arrive while the client is offline. When the client reconnects, the broker delivers the queued messages. This is valuable for power-constrained devices that sleep frequently: they do not miss updates published during their sleep period. QoS 0 messages are fire-and-forget and are *never* queued, even with a persistent session.

What is the benefit of persistent sessions in MQTT, and which QoS levels are affected?	
SA5 When should you choose raw LoRa over LoRaWAN, and when should you prefer LoRaWAN?	Choose **raw LoRa** when you need a custom or proprietary MAC layer, point-to-point links between two specific devices, or full control over duty-cycle scheduling and packet timing. Also appropriate for small closed systems (two to four devices) where the overhead of a full LoRaWAN stack is unjustified. Choose **LoRaWAN** when you need to connect many devices to managed infrastructure, want standardised security (AES-128 OTAA/ABP session keys), need to use an existing public network (e.g., The Things Network), or need the defined device classes (A/B/C) for controlled downlink.
SA6 Why is it hard to benchmark code running in a public cloud environment? Name at least two distinct reasons.	• **Noisy neighbours:** Other tenants' workloads on the same physical host cause unpredictable CPU, cache, and memory-bandwidth contention. Results vary between identical benchmark runs. • **Non-deterministic hardware assignment:** The VM may run on different physical machines across runs (different CPU generations, cache sizes, NUMA topology), making results incomparable without hardware pinning. • **Virtualisation jitter:** The hypervisor's scheduler introduces latency spikes. CPU frequency scaling and thermal throttling are invisible to the tenant. • **Shared network:** Network round-trip times and bandwidth fluctuate with aggregate load from all tenants.
SA7 What is SHACL, and why is it needed in an RDF-based IoT system?	SHACL (Shapes Constraint Language) is a W3C standard for defining validation rules over RDF graphs. For example: "every sensor node must have exactly one `brick:hasUnit` property" or "a room must have at least one `brick:isPartOf` relationship to a floor." It is needed because RDF is schema-free — anyone can add any triple, and without SHACL, downstream queries might silently receive malformed or incomplete data. SHACL acts as the data-quality enforcement layer that rejects or flags non-conforming triples before they enter the system.
SA8 In Elixir's GenServer, what is the difference between `call` and `cast`? Describe the scenario in which using `call` leads to a deadlock.	`call` is synchronous: the caller sends a message to the GenServer and blocks until `handle_call/3` returns `{:reply, value, new_state}`. The caller receives the value. Use it when the result is needed before proceeding. `cast` is asynchronous: the caller sends a message and immediately gets `:ok` back, without waiting. `handle_cast/2` returns `{:noreply, new_state}`. Use it for fire-and-forget updates. **Deadlock:** If a GenServer calls `GenServer.call(self(), :something)` inside one of its own callbacks, it deadlocks. The `call` puts a message in the GenServer's own mailbox and blocks the process waiting for a reply — but the process is blocked and cannot process its own mailbox, so the reply never arrives. The same applies to two GenServers calling each other in a cycle (A calls B, B calls A).

SA9 What is NTP stratum, and what does the difference between stratum 1 and stratum 3 mean in practice?	NTP stratum describes how many hops a clock is from a primary reference source. **Stratum 0** devices are the reference clocks themselves — GPS receivers or atomic clocks — that are not directly on the network. **Stratum 1** servers are machines physically connected to a stratum 0 device; they are typically accurate to within microseconds. **Stratum 3** servers synchronise from stratum 2 servers, which synchronise from stratum 1 servers. Each hop adds propagation delay and jitter; stratum 3 clocks are typically accurate to a few milliseconds. For IoT applications that correlate sensor readings across devices (e.g., matching a temperature spike to a valve event), knowing your NTP stratum tells you the worst-case timestamp uncertainty.
SA10 What is the role of the Brick Schema in building IoT systems, and what kind of query does it enable that a plain sensor database cannot easily answer?	Brick Schema is a standard RDF ontology for describing building systems: sensors, rooms, floors, HVAC zones, and their relationships (`hasPart`, `isLocatedIn`, `feeds`, `hasUnit`). It enables queries that a flat sensor database cannot: for example, "find all temperature sensors in rooms on the south side of floor 3 that feed the Zone-4 HVAC system" — traversing structural relationships rather than filtering by column values. The same Brick model supports applications from thermostat control to GDPR auditing (which sensors monitor which spaces occupied by which people) without changing the underlying data.
SA11 What goes into an MQTT topic string, and what do the `+` and `#` wildcards do?	An MQTT topic is a `/`-separated hierarchy of strings, e.g. `building/floor3/room-U181/temperature`. Hierarchy levels convey structure (building → floor → room → measurement type). `+` matches exactly one level: `building/+/room-U181/temperature` matches any floor. `#` matches zero or more trailing levels: `building/floor3/#` matches all topics under floor 3 (all rooms, all measurement types). `#` may only appear at the end of a subscription string.
SA12 What is flash write endurance, and why does it matter for an edge IoT device that logs sensor data locally?	Flash memory cells can only be erased and rewritten a finite number of times — typically 10,000 to 100,000 write/erase cycles per cell before they fail. Additionally, flash must be erased in large blocks (sectors of 4–64 KB) before individual bytes can be rewritten, so even a 1-byte log entry may require erasing and rewriting an entire sector. For an edge device logging sensor data at 1 Hz continuously, a small flash chip could exhaust its endurance within months if the same sector is overwritten repeatedly. Mitigation strategies include wear levelling (distributing writes across all available sectors), circular log buffers, and reducing log frequency by transmitting instead of storing locally.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
DA1 Explain how noise in sensor sampling interacts with adaptive sampling and the radio budget. How does it cause the budget to be exceeded, and what can be done to mitigate this?	Adaptive sampling adjusts the sampling rate based on how quickly the measured quantity is changing: high rate during rapid change, low rate during stability. This is efficient when the signal truly is the source of variation.

	Noise is a problem because random ADC noise looks identical to real signal change. An adaptive algorithm that interprets noise as genuine change will continuously keep the sampling rate high, even when the physical phenomenon is stable. This generates far more samples than the environment justifies — and correspondingly more radio transmissions. Radio transmission is roughly 100,000–1,000,000× more energy-expensive per bit than a CPU instruction. Each unnecessary sample that gets transmitted represents a disproportionate energy cost. A radio budget set for, say, 10 transmissions per minute will be blown if noise inflates the effective rate to 60 per minute. **Mitigation:** Apply a moving average or median filter to raw ADC values *before* feeding them to the adaptive algorithm. This smooths noise so only genuine environmental changes raise the sampling rate. Add a hysteresis threshold — only increase the rate if the filtered signal changes by more than a noise floor constant (e.g., 2× the measured noise amplitude). This decouples the noise floor from the adaptive trigger and keeps radio transmissions proportional to real physical change, not measurement artefacts.
DA2 Describe the DIKW (Data → Information → Knowledge → Wisdom) hierarchy using a concrete IoT sensor example. Explain what transformation occurs at each step, and why the distinction matters for system design.	Using a humidity sensor in room U181: **Data:** `0.73` — a raw dimensionless number from the ADC output. No unit, no location, no time. Completely meaningless without context. **Information:** "Sensor ID 42, room U181, relative humidity 73%, measured at 14:32:07 on 2026-02-03." Context added: physical unit (%RH), timestamp, location, sensor identity. The reading is now interpretable. **Knowledge:** "Room U181 has been above 70% RH for the past 6 hours. Building standards require <70% RH to prevent mould growth; comfort guidelines recommend 40–60%." Context added: historical trend, domain rules, thresholds, and their implications. The reading is now actionable. **Wisdom:** "The elevated humidity is caused by a faulty dehumidifier last serviced 18 months ago. Given that three ongoing meetings are scheduled in adjacent rooms, the optimal response is to increase ventilation now and schedule maintenance within 48 hours — not an emergency shutdown that would disrupt the meetings." Context added: root cause, business constraints, risk/trade-off reasoning, optimal timing. **Why it matters for design:** A system that stores only raw data accumulates ~500 TB/year for a 10,000-building network — expensive to store and query. A system designed around DIKW processes close to the source: the Edge filters noise, the Fog annotates with context, and only knowledge-level aggregates (5-minute averages, threshold events) are shipped to the Cloud. This drastically reduces bandwidth, storage cost, and query latency.

DA3

Explain the tradeoffs between Edge, Fog, and Cloud processing tiers. For each tier, give one type of processing it is uniquely suited for and justify why.

Edge is the physical device: constrained RAM (256 KB–4 MB), limited CPU, no reliable network dependency. It is uniquely suited for hard real-time processing — e.g., a fire-alarm threshold check must fire within milliseconds regardless of network state, and a network round-trip of 50–200 ms would violate the timing requirement. Edge processing is also the only option that survives network loss (offline capability). It is unsuitable for cross-device aggregation or anything requiring historical data.

Fog is a regional gateway or local server with moderate compute. It is uniquely suited for aggregation across many edge devices without cloud latency — e.g., computing the mean temperature across 50 rooms in a building and triggering floor-level HVAC adjustments within 1–2 seconds. It reduces bandwidth to the cloud by pre-aggregating, and can maintain local models (e.g., occupancy inference) that require data from multiple sensors simultaneously. It is not suitable for large-scale ML training or long-term storage.

Cloud has unlimited compute and storage but 50–300 ms round-trip latency to the edge and requires internet connectivity. It is uniquely suited for training long-horizon ML models (e.g., predicting HVAC failure months in advance using years of historical sensor data), running complex multi-tenant analytics, and serving global dashboards. It is completely unsuitable for anything with sub-second latency requirements.

The guiding principle: process as close to the source as constraints allow. Only push up the tier hierarchy what the lower tier genuinely cannot handle.

DA4

In Elixir, actors can communicate in two ways. What are they, and why should you choose one over the other?

In Elixir, GenServer processes communicate via message passing. There are two patterns:

`GenServer.call/2` sends a message to the server and *blocks the caller* until the server processes it and returns a reply via `handle_call/3` → `{:reply, result, new_state}`. The call is synchronous — the caller knows the operation completed and receives the result. Use `call` when: the caller needs the return value before continuing, or when you need to ensure the operation completed (e.g., reading a registry entry, performing an atomic check-and-modify).

`GenServer.cast/2` sends a message and immediately returns `:ok` to the caller without waiting for any reply. The server handles it via `handle_cast/2` → `{:noreply, new_state}`. Use `cast` when: the caller does not need a result, the operation is a fire-and-forget notification or state update (e.g., logging an event, incrementing a counter), or when you want to avoid blocking the caller and risking timeout.

Key rule: prefer `cast` for updates and events; prefer `call` for queries where the result is needed. Overusing `call` risks timeout under load.

Deadlock with `call`: If a GenServer calls `GenServer.call(self(), :msg)` inside one of its own callbacks, it deadlocks immediately. The `call` blocks the process waiting for a reply — but the process is now blocked and cannot service its own mailbox, so the reply will never arrive. The same applies to circular chains: if A calls B and B attempts to call A, A is blocked waiting for B, and B is blocked waiting for A. Use `cast`, `send`, or `handle_continue` for self-messaging.

DA5

A standard omnidirectional antenna radiates power equally in all directions. In a deployment with one intended receiver, most of that energy is wasted — and it also increases interference with adjacent devices. Beamforming solves this by concentrating radio energy in the direction of the target.

Explain beamforming: what problem it solves, how it works physically, and what benefits it provides for IoT deployments.

How it works: A beamforming system uses an antenna array (two or more antenna elements separated by roughly half a wavelength). The transmitter introduces controlled phase offsets (time delays) between the signals sent by each element. Because of wave superposition, the signals from all elements constructively interfere in the direction of the target (waves add up) and destructively interfere in other directions (waves cancel). The result is a focused "beam" directed at the receiver.

Benefits for IoT:

- **Range:** Concentrating the same transmit power in one direction significantly extends effective range — the same EIRP budget reaches farther.
- **Energy efficiency:** Achieving the same link budget requires less total transmit power, which is critical for battery-powered IoT nodes.
- **Interference reduction:** Devices in non-beam directions receive much weaker signals, reducing co-channel interference in dense deployments.
- **Spatial multiplexing (MU-MIMO):** Multiple independent beams can serve multiple clients simultaneously on the same frequency, increasing network capacity.

Beamforming is standard in 802.11ac/ax (WiFi 5/6) and 5G NR. For IoT it is most relevant in dense smart-building or factory deployments where interference and range are both limiting factors.

DA6

The CAP theorem states that a distributed data system can guarantee at most two of three properties simultaneously: **Consistency** (every read sees the most recent write), **Availability** (every request receives a response), and **Partition Tolerance** (the system continues operating despite network partitions separating nodes).

Explain what the CAP theorem says and what it means for the design of a large-scale IoT time-series storage system.

Since network partitions are a physical reality in any distributed system (including IoT backends), partition tolerance is non-negotiable. The real choice is between Consistency and Availability when a partition occurs.

For a large-scale IoT time-series storage system, the practical choice is typically **AP (Available + Partition-tolerant)**, accepting eventual consistency. Here is why:

	• Sensor readings arrive continuously from thousands of nodes. If we require strict consistency (CP), a network partition could cause write requests to be rejected entirely — meaning we lose readings during the partition period. For time-series data, a lost reading is irrecoverable; a slightly stale read is usually acceptable. • Queries against time-series databases (e.g., "show me the last 24 hours of temperature") tolerate eventually-consistent results because analysts are reviewing historical trends, not making real-time safety decisions based on a write they just performed. • Ingest availability is typically more important than read consistency for IoT: the system must always accept new measurements. The consequence: IoT time-series databases (InfluxDB, TimescaleDB, Cassandra) are typically designed as AP systems. Consistency is sacrificed at partition boundaries and repaired asynchronously. This is acceptable for analytics but means that alerting systems that require strict consistency should use separate, strongly-consistent stores for critical threshold state.

Section 5 — Design Question

QUESTION	MODEL ANSWER
15 A research university wants to deploy a smart monitoring system across **20 greenhouses** on a single campus (roughly 1 km²). Each greenhouse contains **40 sensor nodes**, each measuring temperature, humidity, CO■, and soil moisture. Requirements: • Staff must be **alerted within 5 seconds** if any sensor leaves its safe operating range. • All readings must be **logged to a central database** for later analysis (years of retention). • Researchers need a **web dashboard** to visualise historical trends per greenhouse, per sensor type. It is up to you to identify and argue for a good balance of the involved tradeoffs.	—

Section 3 — Short Answer

QUESTION	MODEL ANSWER
15.1 How frequently should each sensor type sample, and does any measurement type require a higher rate than the others?	Temperature, humidity, CO■, and soil moisture all change slowly relative to human timescales: • **Temperature:** 1-minute intervals are more than sufficient. Thermal time constants in a greenhouse are on the order of minutes. • **Humidity:** 1 minute. • **CO■:** 1 minute; CO■ levels respond to plant photosynthesis and ventilation, which operate on minute timescales. • **Soil moisture:** Every 5–10 minutes is adequate; soil moisture changes over hours, not seconds. None of these measurements require a higher sampling rate. The 5-second alert requirement is a *latency* requirement on the data pipeline, not a sampling frequency requirement — a 1-minute sample can still trigger an alert within 5 seconds of the reading being taken.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER

15.2 What should the network topology look like from sensor node to cloud database, and which network technologies (at the relevant layers of the stack) should be used? Justify your choices.	**Sensor node → Greenhouse gateway (Edge → Fog):** Each greenhouse has 40 nodes within a small space (~30 m²). A low-power mesh protocol is appropriate: **Zigbee** (IEEE 802.15.4, 2.4 GHz) or **Thread** (IPv6 mesh over 802.15.4). Both support ~40–100 nodes per network, have very low power consumption (suitable for battery-powered nodes), and provide reliable mesh routing so nodes do not need line-of-sight to the gateway. Each greenhouse runs one coordinator/border router acting as the Fog gateway. **Greenhouse gateway → Campus server (Fog → local cloud):** With 20 greenhouses across 1 km², a wired **Ethernet/PoE** backbone is the most reliable option. It eliminates RF interference, provides deterministic latency, and each gateway can draw power from the Ethernet cable. If cabling is not feasible, campus **WiFi** (802.11ac/ax) can serve as an alternative, but requires careful RF planning to avoid interference between greenhouse gateways. LoRaWAN would be too bandwidth-limited for 40 sensors × 4 measurements × 60 readings/minute per greenhouse. **Campus server → Cloud database:** Standard **MQTT over TLS** to a cloud broker (e.g., AWS IoT Core or self-hosted Mosquitto), with **QoS 1** to ensure readings survive transient network blips. The cloud backend writes to a time-series database (InfluxDB or TimescaleDB). The web dashboard reads from the same database via a REST or GraphQL API. **Protocol stack summary:** • Sensor → gateway: Zigbee / Thread (IEEE 802.15.4 physical + mesh MAC) • Gateway → campus: Ethernet/IP + MQTT • Campus → cloud: MQTT over TLS/IP

Section 3 — Short Answer

QUESTION	MODEL ANSWER
15.3 Where in the topology should the real-time alert logic run, and why?	The real-time alerting logic (5-second requirement) must run at the **greenhouse gateway (Fog tier)**, not in the cloud. A round-trip to the cloud (sensor → gateway → campus server → cloud → alert) introduces at minimum 500 ms to several seconds of latency depending on network conditions. Cloud processing queuing and broker delays easily add another 5–30 seconds. This violates the 5-second requirement. The gateway receives all 40 nodes' readings from its greenhouse directly via the local mesh, applies threshold rules in-memory, and fires an alert (via MQTT to staff devices or a local alarm) within 1–2 seconds of the reading arriving. The cloud continues to receive all readings for historical storage regardless of whether an alert was triggered. Threshold checking *could* also happen at the sensor node itself (Edge), which would be even faster. However, some alert conditions may involve comparing readings across multiple nodes in the same greenhouse (e.g., "CO■ is high and humidity is also high"), which the individual sensor node cannot do. The gateway is the correct tier.
15.4 Design an MQTT topic hierarchy that supports both per-sensor subscriptions and wildcard subscriptions across all greenhouses. Give examples.	Proposed structure: `greenhouse/{campus_id}/{greenhouse_id}/sensor/{node_id}/{measurement}`

	Examples:
	`greenhouse/odense/gh-07/sensor/node-23/temperature`
	`greenhouse/odense/gh-07/sensor/node-23/humidity`
	`greenhouse/odense/gh-07/sensor/node-23/co2`
	`greenhouse/odense/gh-07/sensor/node-23/soil_moisture`
	
	Useful wildcard subscriptions:
	• All data from greenhouse 7: `greenhouse/odense/gh-07/#`
	• All temperature readings across all greenhouses: `greenhouse/odense/+/sensor/+/temperature`
	• Everything on campus: `greenhouse/odense/#`
	• All CO■ readings campus-wide for the alert service: `greenhouse/+/+/sensor/+/co2`
	
	This hierarchy supports both narrow per-sensor subscriptions (the historian subscribes to specific nodes) and broad cross-greenhouse subscriptions (the alerting service subscribes to all CO■ readings). Adding a new measurement type requires no schema change — just add a new topic level.

Practice Exam 2

SDU · Internet of Things · Lectures 1–12

QUESTION	MODEL ANSWER

Section 1 — Multiple Choice

QUESTION	MODEL ANSWER
MC1 The Elixir pipe operator `|>` passes the result of the left-hand expression as: • A) The last argument to the right-hand function • B) The first argument to the right-hand function • C) A keyword argument named `:input` to the right-hand function • D) A return value that the right-hand function may or may not use	**B** `
MC2 In BLE (Bluetooth Low Energy), which role broadcasts advertising packets so that other devices can discover it? • A) The GATT server • B) The central • C) The peripheral • D) The L2CAP coordinator	**C** In BLE, the **peripheral** advertises; the **central** scans and initiates connections. A GATT server is a role at a higher layer, not the advertising role.
MC3 What is aliasing in the context of signal sampling? • A) Giving a sensor a human-readable name in the metadata model	**B** Aliasing occurs when a signal is sampled below the Nyquist rate; frequencies above the Nyquist limit fold back into the spectrum and appear as false lower-frequency artefacts.

- B) A phenomenon where high-frequency signal components appear as false lower-frequency artefacts after sampling
- C) The process of averaging multiple ADC readings to reduce noise
- D) The mapping of a physical GPIO pin to a software interrupt handler

MC4

B

In Elixir, the pin operator `^` is used to:

`^x` matches against the current binding of `x` without rebinding it. Without `^`, `x = new_value` in a match would simply rebind `x`.

- A) Dereference a pointer to a process mailbox
- B) Match against the *current value* of an already-bound variable rather than rebinding it
- C) Raise a variable to a power
- D) Declare a module-level constant that cannot be rebound

MC5

B

What is the role of the MQTT broker?

The broker is the central message router: it receives PUBLISH packets and forwards them to all clients with matching subscriptions. It does not run on devices, does not encrypt payloads end-to-end, and is not a time-series database.

- A) It runs on each IoT device and manages local topic subscriptions
- B) It is a central server that receives messages from publishers and routes them to matching subscribers
- C) It encrypts MQTT payloads end-to-end between publisher and subscriber
- D) It stores all historical topic data in a time-series database

MC6

B

What distinguishes a LoRaWAN Class C device from a Class A device?

Class C devices keep their receiver open continuously (except during transmission), enabling the gateway to push downlink at any time. Class A opens two brief windows only after each uplink.

- A) Class C uses a higher spreading factor
- B) Class C devices listen continuously for downlink messages (except when transmitting), whereas Class A only opens two short receive windows after each uplink
- C) Class C devices can only communicate with proprietary gateways
- D) Class C requires a licensed frequency band

MC7

A

In a C `struct`, member access through a pointer uses the `->` operator. Why does this operator exist?

`ptr->member` is syntactic sugar for `(*ptr).member` — first dereference the pointer to get the struct, then access the member. No null-check is performed.

- A) It is syntactic sugar for `(*ptr).member` — dereferencing the pointer and then accessing the member
- B) It transfers ownership of the struct to the callee
- C) It performs a null-pointer check before accessing the member
- D) It is equivalent to `&ptr.member` — taking the address of the member

MC8

B

In the DIKW hierarchy, what transformation converts *information* into *knowledge*?

Information → Knowledge requires a domain rule or pattern: "relative humidity >70% promotes mould" converts the fact "73% RH" into actionable knowledge. Adding units/timestamps creates *information* from *data*.

- A) Adding a unit and timestamp to a raw value
- B) Applying a domain rule or pattern that gives the information predictive or explanatory power
- C) Storing the information in a relational database
- D) Transmitting the information to the cloud tier

MC9

B

What is the purpose of SPARQL in an IoT metadata system?

SPARQL (SPARQL Protocol and RDF Query Language) queries RDF graphs using triple-pattern matching. It does not stream sensor data, validate schemas (that is SHACL), or serialise data (that is Turtle/JSON-LD).

- A) Streaming sensor readings in real time over WebSocket
- B) Querying RDF graphs to retrieve resources and relationships that match a pattern
- C) Validating RDF triples against a SHACL schema
- D) Serialising sensor readings into the Turtle RDF format

MC10

C

In FreeRTOS, what happens if a task function reaches its closing `}` and returns normally?

FreeRTOS tasks are designed to never return. The function pointer passed to `xTaskCreate` has no valid return address in the call stack. Returning causes undefined behaviour — typically a jump to address `0` or an unhandled exception. Call `vTaskDelete(NULL)` to terminate.

- A) The task is automatically deleted and its stack freed
- B) The task is suspended until the scheduler restarts it
- C) Undefined behaviour — the return address is invalid, typically causing a crash or hang
- D) The task restarts from the beginning of the function

MC11

B

What is a confounding factor in a performance evaluation experiment?

A confounding factor is an uncontrolled variable that correlates with both the independent variable and the outcome, making it impossible to determine whether the measured effect is caused by the factor under test or the confounder.

- A) A metric that is difficult to measure accurately
- B) An uncontrolled variable that influences the result and makes it hard to attribute the outcome to the factor under test
- C) A statistical outlier that should be removed from the dataset
- D) The target variable whose value you are trying to optimise

MC12

A

In the context of IoT radio communication, what does "duty cycle" refer to?

Duty cycle = time transmitting / total time. Regulatory bodies (e.g., ETSI for EU 868 MHz) cap this to limit interference — e.g., 1% duty cycle means at most 36 seconds of transmission per hour.

- A) The fraction of time a radio transmitter is actively transmitting relative to total time
- B) The number of bits transmitted per second
- C) The ratio of data payload bytes to total packet bytes

• D) The number of retransmissions allowed per message

Section 2 — Multiple Answer

QUESTION	MODEL ANSWER
MA1 Which of the following are characteristics of the Erlang BEAM runtime that make it suitable for concurrent IoT applications? • A) Lightweight processes — millions can run concurrently with a few KB of stack each • B) No shared memory between processes — all communication is via message passing • C) Stop-the-world garbage collection pauses affect the entire VM • D) Per-process garbage collection — one process's GC does not pause others • E) Preemptive scheduling — no single process can starve others	**MA1** A, B, D, E** C is wrong — BEAM uses *per-process* GC, so one process's garbage collection does not pause others. This is a key advantage over JVM-style stop-the-world GC.
MA2 Which are valid consequences of storing *more* data than you have a stated purpose for, in an IoT system subject to GDPR? • A) Violation of the data minimisation principle • B) Violation of the purpose limitation principle if the data is later used for a different purpose • C) Increased operational cost (storage, ingestion, query) • D) Automatic GDPR compliance because more data enables better anonymisation • E) Regulatory penalties if the data includes personal information retained beyond the stated retention period	**MA2** A, B, C, E** D is wrong — storing more data does not automatically improve anonymisation; it typically *increases* re-identification risk. A, B, C, and E are all real consequences under GDPR and operational reality.
MA3 Which wireless technologies operate (at least partially) in the unlicensed 2.4 GHz ISM band? • A) WiFi (IEEE 802.11) • B) Bluetooth / BLE • C) Zigbee (IEEE 802.15.4) • D) NB-IoT • E) LoRa (868 MHz EU band)	**MA3** A, B, C** D is wrong — NB-IoT uses licensed LTE bands. E is wrong — LoRa in the EU uses the 868 MHz ISM band (915 MHz in the US). WiFi, Bluetooth/BLE, and Zigbee all operate in 2.4 GHz.
MA4 Which of the following describe valid use cases for fog-tier processing? • A) Aggregating sensor readings from 50 nodes in a building before forwarding to the cloud, to reduce bandwidth • B) Training a neural network on five years of historical building data	**MA4** A, C, E** B is wrong — training on years of historical data requires cloud-scale compute and storage. D is wrong — serving a global dashboard to hundreds of users is a cloud responsibility. A, C, and E are all genuinely fog-tier tasks (regional aggregation, multi-sensor local inference, local brokering).

• C) Running an occupancy model that requires readings from multiple rooms simultaneously • D) Serving a global analytics dashboard to hundreds of researchers worldwide • E) Local MQTT brokering between devices in the same building without requiring internet access	
MA5 Which statements about Elixir's `GenServer` are correct? • A) `GenServer.call/2` blocks the calling process until a reply is received • B) `GenServer.cast/2` blocks the calling process until the server confirms receipt • C) `handle_call/3` must return a tuple of the form `{:reply, response, new_state}` • D) A GenServer process holds its state in the return value of each callback — there is no mutable variable • E) A GenServer can be registered under a name so it can be looked up without knowing its PID	**MA5** A, C, D, E** B is wrong — `cast` is *asynchronous* and returns `:ok` immediately without waiting for the server to process the message. A, C, D, and E are all correct.
MA6 Which of the following are reasons why the JVM (Java) is unsuitable for Edge-tier IoT devices? • A) The JVM requires 50–200 MB of RAM at startup; most MCUs have 256 KB–4 MB • B) Stop-the-world garbage collector pauses are non-deterministic and can violate real-time constraints • C) Java cannot express bitwise operations • D) JVM cold-start time is measured in seconds, unsuitable for devices that must boot and react in milliseconds • E) Java does not support networking	**MA6** A, B, D** C is wrong — Java fully supports bitwise operations (`&`, `|`, `^`, `<<`, `>>`). E is wrong — Java has extensive networking libraries. A, B, and D are the three core reasons the JVM is unsuitable for MCU deployment.
MA7 Which of the following are valid sources of device identity in an IoT deployment? • A) A hardware serial number burned into the MCU or a dedicated ID chip at manufacture • B) A UUID assigned by the user during physical installation via a mobile app • C) A GPS coordinate hardcoded at deployment time (location-derived identity) • D) The MAC address of the device's radio module • E) The cloud platform always generates identity — devices cannot self-identify	**MA7** A, B, C, D** E is wrong — devices *can* self-identify using hardware serial numbers, manufacturer-burned UUIDs, or GPS-derived location. Cloud-generated identity is one option, not the only one. A, B, C, and D are all valid sources described in the course.

Section 3 — Short Answer

QUESTION	MODEL ANSWER

SA1

What is a C pointer, and what does the dereference operator `*` do when applied to it?

A pointer is a variable that stores a memory address rather than a value directly. Declaring `int *p` means "p holds the address of an int." The dereference operator `*p` follows the address stored in `p` and returns the value stored at that address. Assigning `*p = 42` writes `42` to whatever memory location `p` points to. Pointers are used to pass large data structures to functions without copying them, to build dynamic data structures (linked lists, trees), and to access hardware registers directly at known memory addresses.

SA2

What is the Elixir `Registry` module used for, and what problem does it solve compared to using raw PIDs?

`Registry` is a process name registry: it lets you register a process under a logical name (e.g., `{:room, "U181"}`) and look it up without knowing its PID. The problem it solves: PIDs are ephemeral — if a process crashes and is restarted by a supervisor, its PID changes. Any code holding the old PID can no longer reach it. With Registry, you look up the current PID by name at call time, so restarts are transparent to callers. It supports `unique` mode (one process per key) and `duplicate` mode (many processes per key, for pub-sub fan-out).

SA3

What is the difference between Elixir's `Enum` (eager) and `Stream` (lazy) modules? When would you prefer one over the other?

`Enum` functions process the entire collection immediately, producing an intermediate list at each pipeline step. `Stream` functions are lazy — they compose a description of the transformation and evaluate it one element at a time only when explicitly consumed (e.g., by `Enum.to_list/1`).

Prefer `Enum` when collections are small and you need the full result immediately. Prefer `Stream` when processing large or potentially infinite data sources (files, sensor streams) where holding the entire intermediate collection in memory would be wasteful — `Stream` maintains constant memory regardless of collection size.

SA4

What is the distinction between a "smart" device and an "intelligent" device in the context of this course?

A **smart** device reacts to predefined rules or remote commands. It has network connectivity and programmable state but does not reason about *why* or *whether* it should act. A smart bulb that turns on when told to, or follows a schedule, is smart.

An **intelligent** device reasons, adapts, and learns autonomously without explicit reprogramming. It would model context, infer preferences from past behaviour, and update its behaviour based on experience. The distinction matters because "smart" is a marketing term that conflates connectivity with capability; the course treats intelligence as requiring inference, adaptation, and learning — properties that generally require Fog or Cloud processing beyond what an individual device can supply.

SA5

What does `OPTIONAL MATCH` do in a Cypher query (Neo4j), and why is it necessary in building metadata queries?

`OPTIONAL MATCH` returns all rows from the preceding match even when the optional pattern is absent — absent relationships produce `null` values rather than dropping the row entirely. It is the Cypher equivalent of a SQL `LEFT JOIN`.

It is necessary in building metadata queries because not all rooms have the same sensors installed. A query for "all rooms and their thermostat setpoints" would return zero rows for rooms without a thermostat sensor if a regular `MATCH` were used. `OPTIONAL MATCH` on the thermostat relationship returns all rooms, with `null` for rooms that lack one, enabling a complete picture.

SA6

What is a DMA controller, and how does it benefit a microcontroller collecting ADC samples?

A DMA (Direct Memory Access) controller is a hardware unit that transfers data between peripherals (e.g., an ADC, UART, SPI bus) and memory without involving the CPU for each byte. The CPU initiates the transfer, then continues executing other code; the DMA controller handles the byte-by-byte copying in the background and signals the CPU with an interrupt when the transfer is complete.

For ADC sampling, DMA allows the hardware to continuously fill a buffer with ADC readings at the configured sample rate — even at kilohertz rates — while the CPU processes earlier samples or enters a low-power idle state. Without DMA, the CPU would have to poll or service an interrupt for every single sample, burning CPU cycles and preventing sleep.

SA7

What is the LMT86 temperature sensor's transfer function, and what does it tell you about how to interpret its output?

The LMT86 is an analogue temperature sensor that outputs a voltage proportional to temperature. Its transfer function is approximately linear:

```
V_out (mV) = −10.9 × T(°C) + 1777.3
```

This means the output *decreases* as temperature *increases* (negative temperature coefficient). To convert the raw ADC reading to Celsius, you rearrange the formula: `T = (1777.3 − V_out) / 10.9`. The transfer function is needed because the ADC gives you a dimensionless digital number; without the sensor's transfer function you cannot convert that number to a meaningful temperature in physical units.

SA8

What is "adaptive sampling," and what specific condition should trigger an increase in the sampling rate?

Adaptive sampling dynamically adjusts the sampling rate based on signal behaviour rather than using a fixed rate. The rate should **increase** when the rate of change of the measured signal exceeds a threshold — i.e., when consecutive samples differ by more than a defined amount, indicating the phenomenon is evolving rapidly and a higher rate is needed to track it accurately. The rate should **decrease** when the signal is stable (differences below threshold) to save energy and radio budget. A hysteresis band is commonly added to prevent rapid switching between rates at the threshold boundary.

SA9

In C, what is the difference between passing a struct to a function *by value* and passing it *by pointer*? When should you prefer each?

Passing a struct **by value** copies the entire struct onto the callee's stack. Any modifications inside the function affect only the copy — the caller's original is unchanged. This is safe but expensive for large structs (copying dozens or hundreds of bytes per call).

Passing a struct **by pointer** (`void f(MyStruct *s)`) passes only the address (4 or 8 bytes). The function operates directly on the original struct in memory; modifications are visible to the caller. This is efficient for large structs but requires care — unintended modifications will corrupt the caller's data. Use by-value for small structs where immutability is desired; use by-pointer for large structs or when the function must modify the caller's data.

SA10

What is a LoRaWAN network server, and what two responsibilities does it have that a raw LoRa gateway does not?

The LoRaWAN network server (LNS) sits between the physical gateways and the application server. It has two key responsibilities that a raw LoRa gateway lacks:

	• **De-duplication and routing:** Multiple gateways may receive the same uplink packet (they all hear the same transmission). The LNS deduplicates these copies and forwards only one to the application server. A raw LoRa gateway simply forwards everything it hears. • **MAC layer management:** The LNS handles the LoRaWAN MAC protocol — adaptive data rate (ADR), downlink scheduling, session key management (OTAA/ABP join), and device authentication. A raw LoRa gateway is transparent at the physical layer and has no awareness of these MAC functions.
SA11 What is the Elixir `spawn/1` function, and how does a spawned process communicate its result back to the parent?	`spawn(fn -> ... end)` creates a new lightweight BEAM process that runs the given function concurrently. It returns the PID of the new process immediately — the spawning process does not wait for the child to finish. To communicate the result back, the child explicitly sends a message to the parent's PID: `send(parent_pid, {:result, computed_value})`. The parent calls `receive do {:result, val} -> val end` to collect it. This is the fork-join pattern: spawn N workers, each does computation and sends back a result, the parent collects all results with `receive`. The parent's PID is typically captured in a variable before spawning and passed into the closure.
SA12 In performance evaluation, what is the difference between *accuracy* and *precision* of a measurement? Give an IoT example of a measurement that is precise but inaccurate.	**Accuracy** is how close a measurement is to the true value. **Precision** is how repeatable a measurement is — how tightly clustered repeated measurements are around their mean. A measurement can be precise but inaccurate: a miscalibrated temperature sensor that consistently reads 23.4°C when the true temperature is 20.0°C. Every reading is precise (very little variance between samples) but systematically wrong. In IoT, a common source of this error is an ADC with a reference voltage offset — the readings are stable and repeatable but biased by a fixed error from the true physical value.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
DA1 Describe the BLE protocol stack: what GAP and GATT each define, how they interact, and how a BLE sensor device uses them to expose temperature readings to a mobile app.	BLE's protocol stack separates device discovery/connection (GAP) from data exchange (GATT). **GAP (Generic Access Profile)** defines roles and procedures for devices to find each other and establish connections. The **peripheral** broadcasts small advertising packets at regular intervals (typically 20 ms–10 s) on three dedicated advertising channels. The **central** (e.g., a smartphone) scans for these packets and initiates a connection. GAP also governs security pairing and bonding procedures.

GATT (Generic Attribute Profile) defines how data is structured and exchanged once a connection exists. Data is organised into a hierarchy: **Services** (logical groupings, e.g., "Environmental Sensing Service") contain **Characteristics** (individual data points, e.g., "Temperature Measurement"), each with a UUID, a value, and optional **Descriptors** (metadata about the characteristic, e.g., units, update interval). The **GATT server** (the sensor device) hosts this hierarchy; the **GATT client** (the phone app) reads or subscribes to characteristics.

Interaction for a temperature sensor: The sensor (peripheral) advertises via GAP, announcing it has an Environmental Sensing Service. The phone (central) connects via GAP, then uses GATT to discover the Temperature Characteristic and enable notifications. The sensor sends a GATT NOTIFY packet each time a new reading is taken, without the phone needing to poll. The phone decodes the characteristic value using the GATT specification's standard format (16-bit signed integer, 0.01°C resolution) and displays it.

DA2

Describe Elixir's supervision tree: what a supervisor does, how restart strategies work, and why this architecture enables fault-tolerant IoT backend services without global state corruption.

Elixir's OTP supervision tree is the primary mechanism for building fault-tolerant services. It consists of **supervisor processes** arranged in a tree, with **worker processes** (including GenServers) as leaves.

A **Supervisor** monitors its children. If a child process crashes (raises an unhandled exception, exits abnormally), the supervisor restarts it automatically according to its configured **restart strategy**:

- `:one_for_one` — restart only the failed child; other children are unaffected. Used when workers are independent.
- `:one_for_all` — restart all children when one fails. Used when workers share state that becomes invalid if any one of them fails.
- `:rest_for_one` — restart the failed child and all children started after it. Used when later children depend on earlier ones.

Supervisors also enforce a **max restart frequency** (e.g., 3 crashes in 5 seconds): if a child keeps crashing, the supervisor gives up and itself crashes, propagating the failure up to *its* supervisor. This "let it crash" philosophy means workers never need defensive error-handling for unexpected states — they simply crash and let the supervisor restore a known-good state.

Why this matters for IoT backends: A GenServer handling MQTT ingestion for Room U181 might crash if it receives a malformed payload. With a supervision tree, the crash is isolated — other room processes continue unaffected — and the crashed process is restarted within milliseconds, resuming from its initial state. No global lock is corrupted, no other process is affected. This is the foundation of "nine nines" (99.9999999%) availability in Erlang/Elixir systems.

DA3

Explain the energy budget of an IoT edge device. What are the dominant energy consumers, what order-of-magnitude differences exist between them, and how do duty-cycling and adaptive strategies help stay within budget?

An edge device's energy budget defines how much energy it can consume over its operational lifetime (bounded by battery capacity or energy harvesting rate). The dominant consumers are, in order of magnitude:

	• **Radio transmission:** Roughly 10–100 mA during TX; on the order of 1–10 mJ per packet. This is **100,000–1,000,000× more expensive per bit** than a CPU instruction. • **Radio reception / idle listening:** Still significant — 5–20 mA even while just listening. • **ADC sampling:** Moderate — a few mA for the microseconds of a conversion. • **CPU execution:** 1–10 mA at full speed, but instructions are cheap in energy/operation terms. • **Deep sleep:** Microamps. The dominant strategy for battery life is to be asleep as much as possible. **Duty cycling:** The device spends most of its time in deep sleep, wakes on a timer or interrupt, takes a reading, and potentially transmits — then returns to sleep. If transmission is needed once per minute and the radio is on for 100 ms, the radio duty cycle is 0.17%, drastically reducing average current draw. **Adaptive strategies:** Transmit only when the signal changes meaningfully (adaptive sampling reduces unnecessary transmissions). Aggregate multiple readings into one packet to reduce radio on-time. Use the lowest spreading factor (shortest airtime) that still delivers packets reliably. **Worked example:** 3.7 V, 1200 mAh battery. 20 mA during TX for 200 ms every 60 s → average current from radio = 20 mA × (0.2/60) ≈ 0.067 mA. Adding 3 µA sleep current gives ~0.07 mA average. Battery life ≈ 1200/0.07 ≈ 700 days. But if noise causes adaptive sampling to transmit every 5 s instead, average radio current jumps to ~0.8 mA → battery life drops to ~62 days.
DA4 Explain what confounding factors are in the context of IoT performance evaluation. Give two concrete examples from benchmarking IoT systems, and describe how experimental design can control for them.	A confounding factor is an uncontrolled variable that correlates with the independent variable and influences the measured outcome, making it impossible to attribute the result solely to the factor under test. **Example 1 — Cloud benchmarking.** You are benchmarking two MQTT broker implementations on a public cloud VM. On Tuesday the results favour implementation A. On Wednesday you re-run the same benchmark and implementation B wins. The confound is *noisy neighbours*: other tenants' workloads on the shared physical host changed between runs, altering available CPU bandwidth and cache state. The benchmark is measuring cloud load, not broker performance. Mitigation: run on dedicated (non-shared) hardware, repeat many times and report confidence intervals, or run both implementations simultaneously on separate VMs using the same physical host. **Example 2 — Radio latency on different days.** You measure the end-to-end latency of a LoRa sensor under two duty-cycle configurations. Monday's results look better — but Monday had no other LoRa devices nearby. Wednesday added a neighbour device on the same sub-band, increasing collision probability and retransmissions. The confound is RF environment (channel occupancy). Mitigation: conduct experiments in a controlled RF environment (shielded room or low-activity time window), randomise the order of configurations across repeated trials, and measure channel occupancy as a co-variate.

	General control strategies: randomise trial order (eliminates systematic time-of-day effects), repeat many trials (averages out noise), control all variables except the one under test, measure and report confounders as co-variates, use dedicated hardware rather than shared cloud resources.
DA5 Describe MQTT's publish-subscribe model. How does it differ from a direct request-response model, and what are the architectural implications of topic hierarchy design and QoS level choice for a large IoT deployment?	MQTT decouples producers (publishers) from consumers (subscribers) through a central broker. A publisher sends a message to a **topic string** (e.g., `building/floor3/room-U181/temperature`) without knowing who, if anyone, is subscribed. Subscribers register interest in topic patterns with the broker; the broker routes incoming messages to all matching subscribers. Neither side knows the other exists — this is **spatial decoupling**. This contrasts with a request-response model (e.g., HTTP REST) where the client knows the server's address, sends a request, and waits for a reply. MQTT is also **temporally decoupled** (with persistent sessions): a subscriber that is briefly offline can receive messages published during its absence. **Topic hierarchy implications:** A flat topic structure (`sensor42`) makes it impossible to subscribe to "all temperature sensors on floor 3" without enumerating them. A well-designed hierarchy (`building/{floor}/{room}/{type}`) enables wildcard subscriptions (`building/floor3/#`) that automatically include new rooms added in the future. Poor topic design forces changes to all subscribers when the deployment grows. **QoS implications:** QoS 0 (fire and forget) minimises broker load and latency but drops messages under network failures — acceptable for high-frequency sensor readings where a lost sample is replaced by the next one. QoS 1 (at least once) adds acknowledgment and retransmission — necessary for alerts and commands where no message must be missed, but may deliver duplicates that the consumer must handle idempotently. QoS 2 (exactly once) is reliable and duplicate-free but requires a four-way handshake, doubling the message cost — appropriate for billing events or irreversible actuator commands. Over-using QoS 2 on a high-frequency sensor stream would overload the broker.

Section 5 — Design Question

QUESTION	MODEL ANSWER
16 A hospital wants to deploy an IoT monitoring system across a single ward of **30 patient beds**. Each bed has: • A wearable pulse-oximeter (SpO■ + heart rate, continuous) • A bedside temperature sensor (skin temperature, every 2 minutes) • A nurse-call button (event-driven) Requirements: • **Nurse-call events** must reach the nursing station display within **1 second**. • **Vital sign alerts** (SpO■ < 90% or heart rate outside 40–120 bpm) must reach clinical staff within **3 seconds**. • All readings must be **stored for 7 years** for regulatory compliance. • The system must continue operating if the hospital's internet connection drops.	—

Section 3 — Short Answer

QUESTION	MODEL ANSWER
16.1 What sampling rates are appropriate for the pulse-oximeter and the temperature sensor respectively, and why are they different?	The **pulse-oximeter** (SpO■ + heart rate) should sample at **1–2 Hz** continuously. SpO■ and heart rate can change meaningfully within seconds during a clinical event (e.g., a desaturation episode); a 5-second alert window requires at least one reading per second to detect a threshold breach before the window closes. Higher rates (e.g., 25 Hz for waveform capture) would provide a plethysmographic waveform but are not needed for threshold alerting and would drain the battery far faster. The **bedside temperature sensor** samples every **2 minutes** as specified. Skin temperature changes very slowly (over minutes to hours); high-frequency sampling provides no additional clinical value and wastes energy. They differ because temperature dynamics are slow while SpO■/HR can change clinically fast enough to require second-level resolution to meet the 3-second alert window.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
16.2 What network topology and radio technologies should be used from wearable to nursing station to long-term storage? Justify every choice and explain what happens during an internet outage.	**Wearable → Ward gateway (Edge → Fog):** Each wearable is body-worn and must communicate without wires. **BLE** (Bluetooth Low Energy) is the correct choice: it is designed for body-area networks, is standard in medical-grade wearables, achieves 10–30 m range (sufficient within a single ward room), and its power consumption allows 48-hour battery life with 1 Hz sampling. BLE GATT characteristics map naturally to vital-sign measurements. Each wearable connects to a bedside hub or ward-level gateway acting as a BLE central. **Wearable → Gateway protocol:** BLE GATT notifications for continuous vital signs; BLE advertisements for nurse-call events (fast, no connection setup required for sub-1-second delivery). **Ward gateway → Hospital server (Fog → local cloud):** Wired **Ethernet** from each ward gateway to the hospital's local server. Hospital networks are wired infrastructure; wireless would add latency variability and RF interference near medical equipment (also a safety concern). A local hospital server (on-premises, not cloud) handles alerting and temporary storage. **Hospital server → Long-term storage (local → cloud):** **MQTT over TLS** to a cloud time-series database for 7-year retention. QoS 1 is appropriate — readings must not be lost, but occasional duplicates are acceptable in historical storage. During an internet outage, the local hospital server buffers readings to local disk (the system requirement: must operate during outage) and replays them when connectivity is restored. **Internet outage handling:** The critical safety functions (vital-sign alerts, nurse-call routing) all run locally (ward gateway + hospital server) and do not require internet. Long-term cloud storage is delayed until connectivity resumes; no safety-critical function depends on the cloud.

Section 3 — Short Answer

QUESTION	MODEL ANSWER

16.3 Where in the topology should the vital-sign threshold alerting logic run, and why can it not run in the cloud?	Vital-sign threshold alerting must run at the **ward gateway (Fog tier)** or on the **hospital's local server** — not in the cloud. The reasons: A cloud round-trip (wearable → gateway → internet → cloud → alert → nursing station) adds at minimum 500 ms and in practice several seconds of latency (broker queuing, cloud processing, alert routing back). The 3-second requirement would be violated routinely. Hospital internet links also fail or slow under load. The ward gateway receives BLE notifications directly from wearables with <100 ms latency. Threshold logic running there can trigger an alert on the nursing station display within 1–2 seconds of a reading arriving, meeting the 3-second requirement with margin. The cloud continues receiving all readings for historical storage in parallel, but has no role in real-time alerting.
16.4 The wearable pulse-oximeter is battery-powered and must last at least 48 hours. Identify the dominant energy cost and propose one concrete strategy to extend battery life without missing clinically significant events.	The dominant energy cost is **BLE radio transmission**. At 1 Hz sampling with notifications sent to the gateway, the radio transmits roughly 60 packets per minute. Each BLE notification requires the radio to be active for ~1–5 ms at ~10–15 mA. If the battery is ~150 mAh (typical wearable), radio transmission accounts for the large majority of current draw at rest. **Concrete mitigation: aggregation + burst transmission.** Instead of sending a BLE notification for every individual sample, buffer 5–10 samples in the MCU and transmit them in a single packet every 5–10 seconds. The radio is active for approximately the same time per packet but fires 5–10× less often, reducing average radio current by that factor. SpO■ and heart rate change slowly enough that a 5–10 second delivery latency still meets the 3-second alert window — the alert triggers on the *next* burst if a threshold is breached, worst-case 5–10 seconds after the event. If this latency is unacceptable, a threshold check can run on-device (Edge): if SpO■ drops below 90%, the device immediately transmits a single alert packet bypassing the aggregation buffer.

Practice Exam 3

SDU · Internet of Things · Lectures 1–12

QUESTION	MODEL ANSWER
Section 1 — Multiple Choice	
MC1 What is quantization error in an ADC conversion? • A) The delay between a sample being taken and its value being available in memory • B) The difference between the true analogue input value and the nearest representable digital output value	**B** Quantization maps a continuous analogue value to the nearest of 2^N discrete levels. The error is at most ±½ LSB (least significant bit). It is an inherent property of digitisation, not EMI or floating-point rounding.

- C) The noise introduced by electromagnetic interference on the analogue input pin
- D) The rounding error accumulated when averaging multiple samples in floating point

MC2

B

In C, the `volatile` keyword on a variable declaration tells the compiler to:

`volatile` prevents the compiler from caching the variable in a CPU register across accesses. Without it, the compiler may optimise away repeated reads of a variable it believes hasn't changed — but hardware or an ISR may have changed it in memory.

- A) Store the variable in read-only flash memory instead of RAM
- B) Always read the variable from memory rather than using a cached register copy
- C) Prevent other tasks from accessing the variable concurrently
- D) Align the variable to a 4-byte boundary for faster access

MC3

B

CSMA/CA stands for Carrier Sense Multiple Access with Collision *Avoidance*. How does it differ from CSMA/CD (Collision *Detection*)?

CSMA/CA (wireless): sense the channel, if clear add a random backoff, then transmit — the goal is *avoiding* collisions before they happen. CSMA/CD (wired Ethernet): transmit, detect a collision *during* transmission via voltage monitoring, abort and retry. Collision detection during transmission is impossible in wireless because the transmitter's own signal drowns out incoming signals.

- A) CSMA/CA detects collisions after they happen and retransmits; CSMA/CD prevents them before transmission
- B) CSMA/CA tries to avoid collisions by waiting for the channel to be clear and adding a random backoff before transmitting; CSMA/CD detects a collision during transmission and aborts
- C) CSMA/CD is used in wireless networks; CSMA/CA is used in wired Ethernet
- D) CSMA/CA requires a central coordinator; CSMA/CD is fully distributed

MC4

B

What is a UUID (Universally Unique Identifier) and which version is most commonly used for randomly generated device identities?

UUID v4 is 128 bits of (mostly) random data, formatted as `xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx`. The probability of two random UUIDs colliding is negligible (~10■³■).

- A) A 64-bit sequential counter; version 1
- B) A 128-bit identifier formatted as 8-4-4-4-12 hex digits; version 4 (random)
- C) A 32-bit hash of the device MAC address; version 3
- D) A 256-bit cryptographic key; version 5

MC5

C

In Elixir, what does the `%{}` syntax create?

`%{}` creates a map. `%MyStruct{}` creates a struct (which is a map with a `__struct__` key). Tuples use `{}` and anonymous functions use `fn -> end`.

- A) A tuple with named fields
- B) An anonymous function
- C) A map — a key-value store where keys and values can be any term
- D) A module struct with enforced field types

MC6

What is oversampling in the context of ADC measurement?

- A) Sampling at exactly twice the Nyquist frequency
- B) Taking multiple samples at a rate higher than needed, then averaging them to reduce noise and improve effective resolution
- C) Setting the ADC bit depth higher than the MCU's native word size
- D) Configuring the ADC to sample continuously in the background using DMA

B

Oversampling takes N samples at a rate much higher than necessary and averages them. This reduces random noise (noise averages toward zero) and improves effective resolution. Every factor-of-4 increase in sample count adds approximately 1 bit of effective resolution.

MC7

In LoRaWAN, what does OTAA (Over-The-Air Activation) do?

- A) It updates the device firmware wirelessly
- B) It allows the device to negotiate session keys dynamically with the network server during a join procedure, rather than having them hardcoded
- C) It adjusts the spreading factor dynamically based on signal strength
- D) It enables a LoRaWAN gateway to act as a mesh repeater

B

OTAA performs a cryptographic join handshake between the device and the network server. The device receives NwkSKey and AppSKey dynamically per session. ABP (Activation By Personalisation) hardcodes keys — less secure and less flexible. OTAA has nothing to do with OTA firmware updates or ADR.

MC8

What is the hidden terminal problem in wireless networks?

- A) A device transmits on a hidden (reserved) channel that other devices cannot observe
- B) Two devices are each within range of a common receiver but out of range of each other, causing them to transmit simultaneously and collide at the receiver without either detecting the collision
- C) A device's identity is hidden from neighbouring nodes for security reasons
- D) A routing table entry is hidden from certain nodes to prevent routing loops

B

Classic hidden terminal: device A and device C are both in range of device B (the receiver) but not of each other. A and C cannot sense each other's transmissions, so both decide the channel is clear and transmit simultaneously — causing a collision at B that neither A nor C detects.

MC9

In Elixir, a module attribute defined with `@my_attr value` is:

- A) A mutable global variable shared between all processes in the VM
- B) A compile-time constant evaluated once when the module is compiled
- C) A per-process dictionary entry stored in the process heap
- D) An ETS (Erlang Term Storage) entry accessible across modules

B

Module attributes are evaluated at compile time and embedded in the BEAM bytecode as constants. They are not mutable at runtime, not per-process, and not ETS. Common uses: `@moduledoc`, `@spec`, `@default_timeout 5000`.

MC10

What is the primary purpose of an anti-aliasing filter placed before an ADC?

- A) To amplify weak signals to the full ADC input range
- B) To remove frequency components above the Nyquist frequency before sampling, preventing aliasing
- C) To convert a differential signal to a single-ended signal
- D) To decouple the analogue and digital ground planes

B

An anti-aliasing filter is a low-pass filter that removes frequencies above the Nyquist limit *before* the ADC sees the signal. If omitted, those frequencies fold back into the sampled spectrum as aliasing artefacts that cannot be distinguished from or removed from the genuine signal components.

MC11

In Kconfig (used by ESP-IDF / Linux kernel), what does a `config` entry define?

- A) A CMakeLists variable that controls which source files to compile
- B) A user-configurable compile-time option exposed in `menuconfig`, stored in `sdkconfig`
- C) A runtime configuration parameter loaded from flash at boot
- D) A FreeRTOS task priority that can be set without recompiling

B

Kconfig `config` entries declare named Boolean, integer, or string options. `menuconfig` renders them interactively; the choices are saved to `sdkconfig` and become `CONFIG_*` preprocessor macros in the build. They are compile-time, not runtime, and have nothing to do with task priorities or CMake source lists directly.

MC12

What is dead reckoning for device location estimation?

- A) Estimating position by measuring signal strength from multiple known transmitters
- B) Estimating current position by integrating known velocity and heading from a last-known position, without external reference signals
- C) Using GPS satellites to triangulate position to within 10 metres
- D) Reading a hardcoded GPS coordinate from device flash memory

B

Dead reckoning computes position from a prior known position plus estimated displacement (speed × time, or IMU integration). It accumulates error over time and requires periodic correction from an absolute source. GPS triangulates from satellites.
Signal-strength triangulation is a different technique (trilateration).

Section 2 — Multiple Answer

QUESTION	MODEL ANSWER
MA1 Which of the following are valid inter-task communication mechanisms in FreeRTOS? • A) Queues — for passing data items between tasks • B) Binary semaphores — for signalling that an event has occurred	**MA1** A, B, C, D** E is wrong — tasks are independent execution contexts; you cannot call another task's function directly the way you would a regular function. Direct calls would execute in the *calling* task's context and violate task isolation. A, B, C (queues, binary semaphores, counting semaphores) are the primary mechanisms. D (shared global variables + mutex) is technically valid but less idiomatic than queues.

• C) Counting semaphores — for tracking a pool of available resources • D) Shared global variables protected by a mutex • E) Direct task function calls across task boundaries	
MA2 Which statements about RDF are correct? • A) Every RDF resource is identified by a URI (or blank node) • B) RDF triples can be stored in formats including Turtle, JSON-LD, and RDF/XML • C) RDF requires a schema to be defined before any data can be inserted • D) Multiple RDF graphs from different sources can be merged without naming conflicts, because URIs are globally unique • E) SPARQL is the standard query language for RDF graphs	**MA2** A, B, D, E** C is wrong — RDF is schema-free; triples can be added without a predefined schema (which is *why* SHACL was introduced later). A, B, D, and E are all correct: URIs identify resources, multiple serialisation formats exist, global uniqueness of URIs enables safe merging, and SPARQL is the standard query language.
MA3 Which of the following are valid approaches to reducing quantization error or noise in an ADC measurement? • A) Oversampling and averaging multiple readings • B) Increasing the ADC bit depth (e.g., from 12-bit to 16-bit) • C) Applying a software moving-average filter to successive readings • D) Increasing the ADC clock frequency beyond the Nyquist rate • E) Using a lower attenuation setting to use more of the ADC's input range	**MA3** A, B, C, E** D is wrong — increasing clock frequency beyond the Nyquist rate does not reduce quantization error; it may reduce aliasing if signal bandwidth is the concern, but quantization is a separate issue. A (oversampling + averaging), B (higher bit depth), C (moving average filter), and E (lower attenuation → better use of input range → finer resolution per LSB) are all valid.
MA4 Which properties do Zigbee and Thread share? • A) Both are based on the IEEE 802.15.4 physical and MAC layer • B) Both operate in the 2.4 GHz ISM band • C) Both use IPv6 natively for device addressing • D) Both support mesh networking where devices route for each other • E) Both use the same application-layer protocol (Zigbee Cluster Library)	**MA4** A, B, D** C is wrong — Zigbee does *not* use IPv6 natively; Thread does (Thread is IPv6-first). E is wrong — Thread uses its own application layer (often combined with Matter/OpenThread); the Zigbee Cluster Library is Zigbee-specific. A (both use 802.15.4 PHY/MAC), B (both use 2.4 GHz), and D (both support mesh) are correct.
MA5 Which are valid ways to reduce a compiled binary's size on a flash-constrained edge device?	**MA5** A, B, C**

• A) Compile with size-optimisation flags (`-Os`) • B) Remove unused library components via Kconfig / menuconfig • C) Use `float` instead of `double` for floating-point calculations • D) Replace recursive algorithms with iterative equivalents to reduce stack usage • E) Increase the CPU clock frequency	D is wrong — replacing recursion with iteration reduces *stack usage at runtime* but does not reduce the compiled binary size (the code itself may be similar in size or larger). E is wrong — CPU clock frequency has nothing to do with compiled binary size. A (`-Os` optimises for size), B (removing unused Kconfig components), and C (`float` is 4 bytes vs `double`'s 8 bytes, reducing data segment and library pull-in) are valid.
MA6 Which of the following are part of the six IoT domains described in the course? • A) Data Collection • B) Data Transportation • C) Data Encryption • D) Information / Metadata Management • E) Data Processing • F) Decision Support • G) Actuation	**MA6** A, B, D, E, F, G** C (Data Encryption) is not one of the six domains. The six are: Data Collection, Data Transportation, Information/Metadata Management, Data Processing, Decision Support, and Actuation.

Section 3 — Short Answer

QUESTION	MODEL ANSWER
SA1 What is the `volatile` keyword in C, and give a concrete example of why it is needed in an embedded IoT context.	`volatile` tells the compiler that a variable's value may change at any time outside the normal program flow — e.g., by hardware, by an interrupt service routine, or by another thread. Without `volatile`, an optimising compiler may read the variable into a register once and use the cached value repeatedly, never re-reading memory. This produces incorrect behaviour when the value changes externally. Concrete embedded example: an ISR sets a global flag `uint8_t data_ready = 1;` when a UART byte arrives. The main loop polls `while (!data_ready) {}`. Without `volatile`, the compiler may read `data_ready` once, see 0, and optimise the loop into an infinite loop that never re-reads memory — even after the ISR has set it. `volatile uint8_t data_ready;` forces a memory read on every loop iteration.
SA2 What is an anti-aliasing filter, where in the signal chain must it be placed, and what happens if it is omitted?	An anti-aliasing filter is an analogue low-pass filter that attenuates all frequency components above the Nyquist frequency (half the sample rate). It must be placed in the **analogue signal path before the ADC input** — i.e., in hardware, not in software. Once a signal has been sampled, frequencies that exceeded the Nyquist limit have already been folded back into the spectrum as aliasing artefacts; software cannot remove them because they are now indistinguishable from genuine lower-frequency components. Omitting the filter causes aliased artefacts that permanently contaminate the digital signal.

SA3

What is quantization error, and how does increasing the ADC bit depth reduce it?

An N-bit ADC maps the full input voltage range to 2^N discrete levels. Quantization error is the difference between the true analogue value and the nearest representable level — at most ±½ LSB. For a 12-bit ADC with a 3.3 V range, each LSB = 3.3 / 4096 ≈ 0.81 mV, so the maximum quantization error is ≈ 0.4 mV.

Increasing to 16-bit: LSB = 3.3 / 65536 ≈ 0.050 mV — maximum error ≈ 0.025 mV. Every additional bit halves the maximum quantization error and doubles the resolution. Oversampling can achieve a similar effect in software: averaging 4 samples at 12-bit provides approximately 13-bit effective resolution.

SA4

What is the hidden terminal problem, and how does the RTS/CTS handshake in 802.11 (WiFi) attempt to solve it?

The hidden terminal problem: two devices (A and C) can both reach a shared receiver (B) but cannot hear each other. A senses the channel as idle and begins transmitting; C also senses idle (it cannot hear A) and begins transmitting simultaneously — causing a collision at B.

RTS/CTS addresses this: before transmitting, A sends a short **RTS** (Request to Send) frame to B. B replies with a **CTS** (Clear to Send) frame, which is heard by all devices in B's range — including C. C hears the CTS and knows it must defer. A then transmits its data frame without collision.

The cost: RTS/CTS adds two control frames per data frame, increasing overhead and latency. For small packets the overhead may exceed the payload; most 802.11 implementations only enable RTS/CTS above a configurable packet-size threshold.

SA5

What is a UUID, and what two properties of UUIDs make them well-suited as device identifiers in a large IoT deployment?

A UUID (Universally Unique Identifier) is a 128-bit value formatted as 32 hex digits in groups `8-4-4-4-12` (e.g., `550e8400-e29b-41d4-a716-446655440000`). Two properties make UUIDs well-suited for IoT device identity:

- **Global uniqueness without coordination:** UUID v4 (random) generates identifiers with a collision probability so low (~10■³■) that no central registry is needed. Any device can generate its own UUID offline without checking with a server.
- **Opacity:** UUIDs carry no embedded information about the device's location, manufacturer, or type — preventing reverse-engineering of deployment topology from the identifier alone.

SA6

What is dead reckoning, and when would you use it instead of GPS for location tracking in an IoT device?

Dead reckoning estimates current position by starting from a last known position and integrating motion over time using onboard sensors: an accelerometer or IMU (for acceleration/orientation) or wheel encoder (for speed and direction). No external reference (GPS, cell towers) is needed.

Use dead reckoning instead of GPS when: GPS is unavailable (indoors, tunnels, dense urban canyons where satellite signals are blocked); GPS would consume too much power for a battery-constrained device; or fast position updates are needed between infrequent GPS fixes (GPS receiver takes seconds to acquire a fix). The drawback is error accumulation — IMU drift makes positions increasingly inaccurate over time without periodic correction from an absolute source.

SA7

What is a FreeRTOS queue, how does it differ from a semaphore, and when should you use one over the other?

A **queue** is a FIFO buffer for transferring data items between tasks or between an ISR and a task. Items are copied in and out; the queue holds N items of a fixed declared size. Use a queue when you need to pass actual data (a sensor reading, a command struct) from one task to another.

A **semaphore** (binary or counting) carries no data — it is a signalling mechanism. A binary semaphore has two states (given/taken); a counting semaphore has a count. Use a semaphore when you only need to signal that an event happened (ISR notifying a processing task) or to protect a shared resource via mutual exclusion (mutex). Using a queue to signal events is wasteful; using a semaphore to transfer data is impossible.

SA8

What is Kconfig, and what problem does it solve compared to hardcoding `#define` values in source files?

Kconfig is a configuration system (originally from the Linux kernel, adopted by ESP-IDF and Zephyr) that allows build-time options to be defined in structured `Kconfig` files and configured interactively via `idf.py menuconfig`. The chosen values are saved to `sdkconfig` and generated as `CONFIG_*` preprocessor macros.

The problem it solves: hardcoding `#define BAUD_RATE 115200` in source files requires recompiling after editing source. With Kconfig, options are declared with descriptions, types, defaults, and dependencies; `menuconfig` presents a navigable menu; the build system tracks changes and recompiles only what is affected. This separates configuration from code, enables feature gating (enable/disable entire subsystems), and documents options with their valid ranges.

SA9

What is oversampling, and how does taking N samples and averaging them improve the effective bit resolution of an ADC?

Oversampling takes M samples at a rate much higher than the signal requires and averages them. Averaging reduces random noise because noise is uncorrelated between samples — its expected value after averaging N samples is reduced by a factor of $\sqrt{N}$ in amplitude (N in power).

The improvement in effective bit resolution follows from: each factor of 4× in sample count adds 1 bit of effective resolution. So averaging 4 samples improves a 12-bit ADC to ~13-bit; averaging 16 samples improves it to ~14-bit. This works because averaging shifts the quantization-error floor, effectively interpolating between discrete levels. The trade-off is increased CPU load and slower output rate (or the need for a faster ADC clock).

SA10

What is clock drift in the context of IoT time synchronisation, and what are its practical consequences for correlating sensor readings across devices?

Clock drift is the gradual divergence of a device's local clock from true time, caused by imperfections in the crystal oscillator. Typical MCU crystals drift at 20–100 ppm (parts per million), meaning up to 8.6 seconds per day of error.

Practical consequences for IoT: if two sensor nodes each measure a temperature event but have unsynchronised clocks drifted by 5 seconds, fusing their readings in time-series analysis places them at different timestamps even though they are simultaneous. Anomaly detection, cross-device correlation, and causal analysis all become unreliable. Over long deployments, drift accumulates — a device without NTP may be minutes off after weeks. Mitigation: synchronise to NTP on every reconnection; for offline devices, record the NTP offset at last sync and apply it as a correction.

SA11
 	
What is the Elixir `case` expression, and how does it differ from an `if` expression?
	`case` matches an expression against a sequence of patterns (optionally with guard clauses) and evaluates the first matching branch. It is exhaustive pattern matching on a value:
	
	`case result do`
	`{:ok, value} -> process(value)`
	`{:error, reason} -> log(reason)`
	`end`
	
	`if` tests a single Boolean condition and has an optional `else`. Both are expressions in Elixir (they return a value). `case` is preferred when the number of alternatives exceeds two, when destructuring is needed, or when the branches depend on the *shape* of a value rather than a simple true/false test. `if` is idiomatic only for simple binary conditions.

SA12 In the context of radio communication, what is the difference between a unicast, a broadcast, and a multicast transmission?	**Unicast:** a transmission addressed to exactly one recipient device. The frame contains the destination's specific address. All other devices on the medium discard it at the MAC layer. **Broadcast:** a transmission addressed to all devices on the network segment simultaneously. Every device receives and processes it. Excessive broadcast traffic can flood a network (broadcast storm). **Multicast:** a transmission addressed to a defined group of devices that have explicitly subscribed to a multicast group address. Only group members process the frame. More efficient than sending N unicasts to N recipients but less disruptive than a full broadcast. Used in MQTT over WiFi for group subscriptions, and in LoRaWAN for Class B downlink slots.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
DA1 Explain the hidden terminal problem in wireless networks. Describe how CSMA/CA detects channel state, why the hidden terminal problem causes CSMA/CA to fail, and how the RTS/CTS mechanism addresses it. What is the cost of using RTS/CTS?	**The problem:** CSMA requires each node to sense the channel before transmitting. If the channel is busy, the node defers. This works when all nodes can hear each other. The hidden terminal problem arises when two nodes (A and C) are both in range of a common receiver (B) but cannot hear each other's transmissions. A senses the channel idle (it cannot hear C), begins transmitting to B. Simultaneously C also senses idle (it cannot hear A), begins transmitting to B. A collision occurs at B, but neither A nor C detects it — they both believe their transmission was successful. Without an acknowledgment, A and C must time out and retry, repeatedly colliding. **How CSMA/CA alone fails:** The random backoff in CSMA/CA helps when nodes *can* hear each other (they both wait, one's backoff expires first, it transmits, the other defers). But if A and C cannot hear each other, they independently draw random backoffs, one happens to be shorter, that node transmits — and the other node, still unable to hear the transmission, fires at any point during it.

	RTS/CTS mechanism: Before a data frame, A sends a small RTS (Request to Send) frame to B. B responds with CTS (Clear to Send), which it broadcasts into B's full coverage area. C, being in B's range, hears the CTS even though it cannot hear A. The CTS contains the expected duration of the upcoming data transfer. C sets a `NAV` (Network Allocation Vector) timer and defers for that duration — effectively silencing itself without having heard A at all. A proceeds to transmit collision-free. **Cost of RTS/CTS:** Two additional control frames per data exchange. For small payloads (e.g., a 10-byte sensor reading), the RTS+CTS overhead (typically 20–40 bytes plus SIFS/DIFS gaps) may exceed the payload itself, nearly doubling channel time. For this reason most 802.11 implementations disable RTS/CTS for packets below a threshold (e.g., 2346 bytes by default) and enable it only for large frames where the collision cost justifies the overhead.

DA2
 	
Explain Elixir's pattern matching system in depth. Cover: the match operator, destructuring tuples and maps, the pin operator, guard clauses in function heads, and how multiple function clauses together implement conditional dispatch without `if/case`.
	Pattern matching is the foundation of Elixir's control flow. The `=` operator is not assignment — it attempts to *match* the right-hand side against the left-hand pattern, binding variables as needed.
	
	Destructuring: `{a, b, c} = {1, 2, 3}` binds `a=1`, `b=2`, `c=3`. `[head | tail] = [1, 2, 3]` binds `head=1`, `tail=[2,3]`. Nested structures: `%{user: %{name: name}} = response` extracts a name from a nested map. If the pattern doesn't match, a `MatchError` is raised — this is intentional; unexpected shapes are bugs, not edge cases to silently handle.
	
	Pin operator `^`: Once a variable is bound, re-using its name in a match would rebind it. `^x` pins `x` to its current value and matches only if the right-hand side equals that value. Example: `^expected = actual` fails (raises MatchError) if `actual ≠ expected`.
	
	Guard clauses: `when` clauses in function heads add Boolean conditions beyond structural shape. Only a limited set of expressions are allowed in guards (comparison operators, `is_integer/1`, `is_binary/1`, arithmetic, etc.) — side effects are forbidden because guards may be evaluated multiple times.
	
	Multiple function clauses: Instead of a single function with an `if/case` tree, Elixir functions can have multiple clauses matched top-to-bottom:
	
	`def describe({:ok, val}), do: "success: #{val}"`
	`def describe({:error, msg}), do: "error: #{msg}"`
	`def describe(other), do: "unknown: #{inspect other}"`
	
	The BEAM matches pattern and guard simultaneously for each clause, evaluating the first that succeeds. This eliminates nested conditionals and makes each case explicit, independently testable, and readable.

DA3 Explain how time synchronisation works in an IoT system. Cover NTP operation and stratum, clock drift at the edge, the practical consequences of unsynchronised clocks for sensor data correlation, and what to do when NTP is unavailable (e.g., during network outage).	**NTP operation:** NTP (Network Time Protocol) synchronises clocks by exchanging timestamps with a server. The client measures the round-trip delay, estimates one-way latency, and adjusts its clock accordingly. The accuracy of the correction depends on symmetric network delay; asymmetric paths introduce offset errors.

	NTP stratum: Stratum 1 servers are directly connected to stratum 0 reference clocks (GPS, atomic). Stratum 2 synchronise from stratum 1, stratum 3 from stratum 2. Each hop adds uncertainty — stratum 3 devices are typically accurate to a few milliseconds; stratum 1 to microseconds. **Clock drift at the edge:** MCU crystal oscillators drift 20–100 ppm. At 50 ppm, a device drifts 4.3 seconds per day. Without periodic NTP correction, timestamps become unreliable. In a dense deployment, two drifting devices accumulate independent errors — their clocks may diverge from each other by seconds per day even if both started synchronised. **Consequences for sensor data:** Correlating events across devices requires timestamps to be trustworthy. A 5-second clock error means an event recorded at device A at T=100 and at device B at T=105 may actually be simultaneous — or may be genuinely 5 seconds apart. Anomaly detection, causal analysis, and data fusion all depend on temporal alignment. A 5-second error is the difference between "simultaneous pressure spike in three pipes" (a water-hammer event) and "three unrelated pressure events." **NTP unavailable:** During network outages, devices must use local time. Strategies: (1) record the NTP offset at the last successful sync and apply it as a known-good correction for subsequent timestamps; (2) note the elapsed time since last sync and flag timestamps with an uncertainty estimate that grows with time since sync; (3) use a hardware RTC (Real-Time Clock) module with a battery-backed crystal, which drifts more slowly than a bare MCU oscillator and retains time across power cycles.
DA4 Describe the COP (Consistency, Overhead, Performance) evaluation model for IoT systems. What does each dimension represent, why can you not optimise all three simultaneously, and how should this guide the design of a performance evaluation experiment?	The COP evaluation model states that for any IoT or distributed system, you can trade off three properties but cannot simultaneously optimise all of them: **Consistency (C):** The degree to which measurements and system state are accurate and free from noise, quantization error, or logical inconsistency. Improving consistency typically requires more samples (oversampling), better sensors (more expensive), or more processing (filtering, calibration). **Overhead (O):** The total resource cost of the evaluation or system operation — energy, bandwidth, CPU time, memory. Reducing overhead means faster, simpler, or less frequent operations. **Performance (P):** The quality of the outcome — throughput, latency, accuracy of prediction, event-detection reliability. High performance usually requires more resources and/or more consistent inputs. **The tension:** Improving C (more samples, better filtering) increases O (more ADC reads, more computation, more radio transmissions). Improving P (detecting events faster) reduces opportunities to save O (must sample and transmit more frequently). Reducing O (sleep longer, transmit less) degrades both C (fewer samples, more quantization noise) and P (slower event detection, higher alert latency).

Experimental design implication: When benchmarking an IoT system, changing one factor inevitably moves at least one other. A fair evaluation must hold two of the three fixed and vary only the factor under test — otherwise the "improvement" in one dimension may simply be the result of relaxing another. For example, claiming "our new protocol improves throughput" without noting that it uses 3× more energy is misleading unless O is held constant.

DA5

Explain the six IoT domains and give a concrete example of each for a smart building energy management system. Explain why the domains must be traversed in order and what happens if one domain is poorly designed.

The six domains form a strict pipeline; each depends on the output of the previous.

1. Data Collection: Physical sensing of the environment. Example: DS18B20 digital temperature sensors on each floor reading every 30 seconds; CO■ sensors and PIR occupancy detectors in each room. Poor design here (wrong sensor, bad placement, too-low sampling rate) corrupts everything downstream — garbage in, garbage out.

2. Data Transportation: Moving raw readings from sensor to the next tier. Example: Sensors publish over MQTT to a local Zigbee-to-Ethernet gateway; the gateway forwards to a cloud broker over TLS/TCP. A poorly designed transport layer loses messages (QoS 0 under a congested WiFi network) or introduces latency that makes real-time alerting impossible.

3. Information / Metadata Management: Giving raw values context. Example: An RDF Brick Schema model maps sensor IDs to rooms, floors, and buildings; associates units (Kelvin, ppm) and calibration offsets; links occupancy PIR sensors to HVAC zones. Without this, `sensor_17 = 0.73` is meaningless to any downstream consumer.

4. Data Processing: Transforming contextualised readings into derived quantities. Example: A moving average smooths noisy temperature readings; an occupancy model infers whether a room is occupied from PIR readings over the past 10 minutes; absolute humidity is computed from temperature + relative humidity. A poorly designed processing step (e.g., no noise filtering) causes the Decision Support layer to fire false alarms.

5. Decision Support: Presenting processed information for human or automated decisions. Example: A dashboard shows per-zone temperature vs. setpoint with colour coding; an alert fires when CO■ exceeds 1000 ppm. A well-designed Decision Support layer presents only actionable information — too many alerts causes alert fatigue.

6. Actuation: Acting on the physical world based on decisions. Example: A HVAC controller adjusts valve positions via Modbus; a relay toggles the heating element on/off. Actuation closes the feedback loop. A poorly designed actuation layer (wrong protocol, no acknowledgment, actuator stuck) means decisions are never translated into physical change.

Why order matters: You cannot actuate without a decision; you cannot make a decision without processed data; you cannot process without transported data; you cannot transport without collected data. Skipping or poorly implementing any domain cascades failures through all subsequent ones.

Section 5 — Design Question

QUESTION	MODEL ANSWER
17 A city council wants to deploy IoT sensors across **500 public waste bins** spread over a city of 10 km². Each bin has an ultrasonic fill-level sensor. The system should: • **Report bin fill level** to a central platform every 15 minutes (routine). • **Send an immediate alert** when a bin exceeds 90% full, so a collection truck can be dispatched within 2 hours. • **Optimise truck routes** daily using historical fill data (which bins fill fastest, at what times of day). The bins are powered by a **small solar panel with a 4000 mAh battery backup**. There is no wired infrastructure — all communication must be wireless.	—

Section 3 — Short Answer

QUESTION	MODEL ANSWER
17.1 Which radio technology is most appropriate for the sensor-to-network link, and why? Discuss at least two alternatives and explain why you reject them.	**Best choice: LoRaWAN.** The bins are spread across 10 km² with no wired infrastructure. LoRaWAN's key advantages here: long range (a single gateway covers several km in an urban environment), very low power consumption (a LoRa transmission takes ~100–500 ms and the radio sleeps otherwise — enabling years of operation on a small battery), and suitability for low-data-rate, infrequent reporting (a fill-level reading is tens of bytes, once per 15 minutes). Public LoRaWAN infrastructure (TTN or municipal gateways) may already exist. **Why not WiFi (802.11):** WiFi requires the bin to connect to an access point. Street infrastructure has no WiFi coverage. Even if hotspots existed, WiFi consumes ~150–300 mA during connection setup — incompatible with a solar/battery budget at this scale. **Why not BLE:** BLE has a range of 10–30 m outdoors. Covering 500 bins across 10 km² would require thousands of relay nodes or gateways. It is designed for personal-area networks, not city-scale deployment. **Why not cellular (NB-IoT):** NB-IoT is technically viable (low power, long range, uses licensed spectrum) and is a reasonable alternative if a municipal SIM contract is available. The tradeoff: NB-IoT has recurring SIM costs and carrier dependency; LoRaWAN can use free public infrastructure or a one-time private gateway deployment.
17.2 The fill-level sensor runs on the same battery as the radio. Describe the duty-cycle strategy you would use to meet the 15-minute reporting interval while maximising battery life.	The bin should spend almost all its time in **deep sleep**. The duty-cycle strategy: • Wake from deep sleep on a hardware timer every 15 minutes. • Power up the ultrasonic sensor, take a fill-level reading (takes ~50–100 ms). • Compare to 90% threshold locally on the MCU. • If below 90%: transmit one LoRa packet with the fill level. Radio on-time ≈ 100–500 ms depending on SF. • If above 90%: transmit an immediate alert packet (higher priority).

	• Return to deep sleep. At SF9 (moderate range/data-rate balance) a packet takes ~185 ms. 15-minute interval: radio duty cycle = 185 ms / 900,000 ms = 0.02%. Average current from radio ≈ 20 mA × 0.0002 = 0.004 mA. Deep sleep current for an ESP32 ≈ 10–20 µA. Total average ≈ 0.02–0.03 mA — a 4000 mAh battery lasts ~6–8 years even without solar. In practice solar tops up the battery daily, making lifetime effectively indefinite.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
17.3 Design the data pipeline from the sensor to the route-optimisation engine. What are the processing steps, where does each step run (Edge / Fog / Cloud), and how is the immediate 90%-full alert delivered reliably despite the constrained radio?	**Step 1 — Edge (on the bin MCU):** Read the ultrasonic sensor, convert distance to fill percentage, apply threshold check. Transmit one LoRa packet: `{bin_id, fill_pct, timestamp, battery_voltage}`. The edge does no further processing — it is too constrained, and aggregation requires data from all bins. **Step 2 — Fog (LoRaWAN gateway + local network server):** The gateway receives the radio packet and forwards it over IP to the LoRaWAN network server. The network server de-duplicates packets (multiple gateways may receive the same transmission), decrypts the payload, and forwards the decoded reading to the cloud application server via MQTT or HTTP. The network server also manages downlink scheduling for any control messages. **Step 3 — Cloud (application server + time-series database):** The application server receives readings, writes them to a time-series database (e.g., InfluxDB or TimescaleDB) with bin_id, timestamp, and fill_pct. This is the authoritative record for all historical data. **Step 4 — Alert path:** The application server applies the 90% threshold on every ingested reading. If exceeded, it publishes an alert to an MQTT topic `waste/alert/{bin_id}`, which triggers a push notification to dispatch software and the truck driver's mobile app. LoRaWAN is inherently reliable enough for this: QoS 1 in the LoRaWAN MAC ensures the packet is acknowledged by the gateway. If a packet is lost, the bin retransmits on the next wake cycle (15 minutes later) — within the 2-hour dispatch window. **Step 5 — Route optimisation (Cloud batch job):** A nightly batch job reads historical fill data from the time-series database, builds a model of fill rate per bin per time-of-day, and generates the next day's optimal collection route (minimising truck distance while ensuring no bin exceeds capacity). This is pure cloud analytics — no latency requirement, requires months of historical data.

Section 3 — Short Answer

QUESTION	MODEL ANSWER
17.4 Over 10 years, each bin transmits a fill-level reading every 15 minutes. Estimate the total data volume for the entire 500-bin fleet and argue whether you should store raw readings or aggregated statistics for the route-optimisation use case.	Each reading: approximately 20 bytes (bin_id: 4 bytes, fill_pct: 1 byte, timestamp: 4 bytes, battery: 2 bytes, overhead: ~9 bytes). Per bin per year: (60/15) × 24 × 365 = 35,040 readings × 20 bytes = **700 KB/bin/year**.

	500 bins × 700 KB = **350 MB/year** for the whole fleet. Over 10 years: **3.5 GB total**. 3.5 GB is trivially small by modern standards — a single hard disk. Raw readings should be stored in full. The argument for raw storage: route optimisation requires detailed temporal patterns (which hours fill fastest, weekday vs. weekend differences, seasonal trends). Aggregating to, say, daily averages would destroy the hourly signal needed to optimise truck departure times. Storage cost at 3.5 GB/decade is effectively zero compared to the operational cost of the sensor network. The GDPR data minimisation argument does not apply here: fill-level readings from a public waste bin are not personal data (they describe the bin, not a person), so there is no regulatory pressure to aggregate or delete.

Practice Exam 4

SDU · Internet of Things · Lectures 1–12

QUESTION	MODEL ANSWER

Section 1 — Multiple Choice

QUESTION	MODEL ANSWER
MC1 In C, a `static` variable declared inside a function: • A) Is placed on the stack and deallocated when the function returns • B) Is initialised every time the function is called • C) Retains its value between calls and is initialised only once • D) Is visible to all functions in the translation unit	**C** A `static` local variable is allocated in the BSS/data segment, not the stack. It is zero-initialised (or explicitly initialised) only once at program start and retains its value across calls. A is the behaviour of non-static locals; B is wrong; D is wrong (it has function scope).
MC2 What does the MQTT `retain` flag on a published message do? • A) It prevents the broker from forwarding the message to any subscriber • B) It causes the broker to store the last retained message for a topic and deliver it immediately to new subscribers • C) It tells the broker to keep the message in a queue until the publisher confirms delivery • D) It sets the QoS level to 2 (exactly-once delivery)	**B** The `retain` flag tells the broker to keep the last message published to that topic and push it immediately to any client that subscribes later. This means a newly connected subscriber sees the current state without waiting for the next publish. A, C, D are all wrong.
MC3 What is 6LoWPAN?	**B** 6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) is an IETF adaptation layer (RFC 4944 / RFC 6282) that compresses IPv6 headers from 40 bytes down to a few bytes so they fit in 802.15.4's 127-byte max frame. A, C, D misidentify it.

- A) A LoRaWAN variant that supports IPv6 addressing
- B) An adaptation layer that compresses IPv6 headers to fit inside IEEE 802.15.4 frames (≤127 bytes)
- C) A 6 GHz band extension of the LoRa PHY
- D) A 6-byte addressing scheme for BLE peripherals

MC4

B

In Elixir, the `with` expression is used to:

`with` evaluates a series of match clauses left-to-right; if any clause fails to match, execution jumps to the `else` block. It is used to flatten nested case/ok/error chains. A, C, D are unrelated.

- A) Define a module-level constant visible to all functions
- B) Chain multiple pattern matches and short-circuit to an `else` clause on the first mismatch
- C) Spawn a linked child process that shares the parent's state
- D) Import all public functions from another module into the current scope

MC5

B

In LoRa, increasing the spreading factor (SF) from SF7 to SF12:

Each step up in SF doubles the number of chips per symbol, halving the bit rate (e.g., SF7 ≈ 5.5 kbps vs SF12 ≈ 250 bps at BW 125 kHz). The longer symbol time improves sensitivity by ~2.5 dB per SF step, extending range. Time-on-air grows exponentially with SF. A is backwards; C and D are wrong.

- A) Increases the data rate and reduces range
- B) Reduces the data rate but increases link budget, time-on-air, and energy per message
- C) Has no effect on time-on-air; only frequency matters
- D) Enables frequency hopping across multiple sub-bands

MC6

B

What is a watchdog timer in an embedded system?

A watchdog is a countdown timer in hardware. Software must periodically write a "kick" value to prevent it reaching zero; if it does, the MCU resets. This recovers from infinite loops, deadlocks, or stack overflows. A, C, D are wrong.

- A) A timer that generates an interrupt every millisecond for the RTOS scheduler
- B) A hardware timer that resets the MCU if software fails to periodically "kick" it, recovering from hangs
- C) A DMA channel that monitors ADC overflow conditions
- D) A software counter that tracks elapsed time since last GPS fix

MC7

B

What is Adaptive Data Rate (ADR) in LoRaWAN?

LoRaWAN's network server monitors link quality and instructs devices to use the fastest SF and lowest TX power that still maintains the link, reducing ToA and saving battery. A, C, D are wrong.

- A) A mechanism that dynamically adjusts the MQTT QoS level based on link quality
- B) A network-server mechanism that adjusts a device's spreading factor and transmit power to optimise data rate while respecting duty-cycle limits
- C) A protocol that switches between LoRa and NB-IoT depending on signal strength
- D) A retransmission scheme for Class B devices

QUESTION	MODEL ANSWER
MC8 In Elixir, how does `Task.async/1` + `Task.await/2` differ from bare `spawn/1`? • A) `Task.async` creates a named process; `spawn` creates an anonymous one • B) `Task.async` links the task to the caller, propagates crashes, and returns a value via `Task.await`; `spawn` is fire-and-forget with no built-in result return • C) `Task.async` runs the function synchronously in the caller's process • D) `spawn` is supervised; `Task.async` is not	**B** `Task.async` sets up a monitor link so a crash in the task propagates to the caller; `Task.await` blocks until the result is returned or timeout occurs. `spawn` is truly fire-and-forget: no result, no crash propagation. A, C, D are wrong.

Section 2 — Multiple Answer

QUESTION	MODEL ANSWER
MA1 Which of the following are characteristics of NB-IoT that distinguish it from LoRaWAN? • A) NB-IoT uses licensed cellular spectrum; LoRaWAN uses unlicensed ISM bands • B) NB-IoT provides guaranteed QoS via 3GPP standards; LoRaWAN is best-effort • C) NB-IoT supports bidirectional communication with low latency downlink; LoRaWAN Class A has constrained downlink windows • D) NB-IoT requires a SIM card and mobile network subscription; LoRaWAN does not • E) Both NB-IoT and LoRaWAN are equally power-efficient for all duty cycles	**MA1** A is correct: NB-IoT operates in licensed LTE bands (e.g., 700–900 MHz); LoRaWAN uses unlicensed ISM. B is correct: NB-IoT inherits 3GPP QoS mechanisms, ACKs, and retransmissions; LoRaWAN is best-effort with no guaranteed delivery (Class A). C is correct: NB-IoT supports PSM and eDRX but can receive downlink at any time when awake; LoRaWAN Class A only opens two short RX windows after each uplink. D is correct: NB-IoT requires a SIM and carrier agreement; LoRaWAN requires a gateway but no subscription. E is wrong: NB-IoT is generally better for latency-sensitive or higher-throughput use cases; LoRaWAN is better for very low duty-cycle, battery-constrained nodes.
MA2 Which of the following are valid Elixir error-handling patterns? • A) `{:ok, value} / {:error, reason}` tagged tuples from function returns • B) `try / rescue` for catching exceptions raised by `raise/1` • C) `catch` for catching thrown values from `throw/1`	**MA2** A is correct: the idiomatic Elixir pattern is to return `{:ok, value}` or `{:error, reason}` and pattern-match on the result. B is correct: `try/rescue` catches exceptions raised with `raise`. C is correct: `catch` catches values thrown with `throw` (used for non-local returns). D is correct: supervision trees are the primary fault-tolerance mechanism — let a process crash and the supervisor restarts it. E is wrong: using `nil` as a universal error sentinel is not idiomatic; it discards the reason and leads to silent failures.

• D) Supervision trees that restart crashed processes rather than handling errors inline • E) Using `nil` as a universal error sentinel (same as in Erlang)	
MA3 Which parameters increase LoRa time-on-air (and therefore energy per transmission)? • A) Larger spreading factor (e.g., SF12 vs SF7) • B) Wider bandwidth (e.g., 500 kHz vs 125 kHz) • C) Lower coding rate (e.g., 4/8 vs 4/5) • D) Longer payload (more bytes) • E) Higher transmit power (dBm)	**MA3** A is correct: higher SF increases the number of chips per symbol, directly increasing ToA (approximately doubles per SF step). B is wrong: wider bandwidth shortens ToA (more chips transmitted per second). C is correct: lower coding rate means more redundancy bits per payload bit (4/8 = 50% overhead vs 4/5 = 20%), increasing ToA. D is correct: more payload bytes directly add symbol time. E is wrong: TX power affects link budget and energy per unit time but does not change ToA (it changes total energy only because the radio draws more current at higher power).
MA4 Which of the following are valid RDF serialisation formats? • A) Turtle (`.ttl`) • B) JSON-LD (`.jsonld`) • C) RDF/XML (`.rdf`) • D) N-Triples (`.nt`) • E) CSV with subject/predicate/object columns	**MA4** A is correct: Turtle is the most human-readable RDF format. B is correct: JSON-LD is an RDF serialisation that is also valid JSON. C is correct: RDF/XML is the original W3C serialisation format. D is correct: N-Triples is a line-based, easily parsed format. E is wrong: plain CSV with s/p/o columns is not a standard RDF format; it has no URI syntax, prefix mechanism, or datatype support.

Section 3 — Short Answer

QUESTION	MODEL ANSWER
SA1 What is a watchdog timer, and give a concrete example of when it would save an IoT device from requiring a manual reset?	A watchdog timer is a hardware countdown register that resets the MCU if software does not "kick" it (write a reset value) within a configured interval. If the firmware enters an infinite loop, deadlocks on a mutex, or hangs waiting for a peripheral, the kick never comes and the MCU reboots automatically. Concrete example: an IoT weather station's I²C sensor driver hangs waiting for an ACK that never arrives (sensor unplugged or bus stuck). Without a watchdog, the device is frozen permanently. With a 10-second watchdog, it reboots, re-initialises the bus, and resumes reporting.
SA2	When a message is published with `retain=true`, the broker stores it as the "last known good" value for that topic. Any client that subscribes to the topic later receives this retained message immediately, without waiting for the next publish. This is valuable for intermittently-connected IoT devices: a dashboard that reconnects after a network outage immediately sees the current sensor state rather than showing "unknown" until the device next transmits.

What does the MQTT `retain` flag do, and why is it useful for an IoT device that connects intermittently?

SA3

6LoWPAN is an IETF adaptation layer (RFC 4944, RFC 6282) that sits between IPv6 and IEEE 802.15.4. It is needed because 802.15.4 has a maximum frame size of 127 bytes, whereas an uncompressed IPv6 header alone is 40 bytes, leaving only ~87 bytes for payload after MAC overhead. 6LoWPAN compresses the IPv6 header (often to 2–3 bytes using context-based compression), fragments large IPv6 datagrams across multiple 802.15.4 frames, and reassembles them, making it possible to use standard IPv6 protocols over constrained mesh networks.

What is 6LoWPAN, and why is an adaptation layer needed between IPv6 and IEEE 802.15.4?

SA4

`with` chains a series of `pattern <- expression` clauses. If every clause matches, the final `do` block executes with all bound variables in scope. If any clause fails to match, the `else` block receives the non-matching value. This avoids the "pyramid of doom" — deeply nested `case {:ok, x} ->` blocks — that grows one indent level per fallible step. Instead, the happy path reads as a flat sequence of steps, and error handling is collected in one `else` block.

What is the Elixir `with` expression, and how does it improve readability over deeply nested `case` expressions?

SA5

Time-on-air (ToA) is the duration a LoRa packet occupies the radio channel. It depends on SF, bandwidth, coding rate, and payload length — at SF12, BW 125 kHz, a 20-byte packet takes roughly 1.5–2 seconds. The EU 868 MHz ISM band imposes a 1% duty-cycle limit: per 100 seconds, a device may transmit at most 1 second. If a single transmission takes 2 s, the mandatory off-time is at least 200 s (≈3.3 minutes), regardless of application requirements. This forces designers to either reduce SF, shorten payloads, or space messages further apart.

What is LoRa time-on-air, and why does it matter for a device subject to the 1% duty-cycle limit in the EU 868 MHz ISM band?

SA6

`xTaskCreate` allocates the task stack and TCB (Task Control Block) from the FreeRTOS heap at runtime; if the heap is exhausted it returns `pdFAIL`. `xTaskCreateStatic` takes caller-provided buffers for both the stack and TCB, so no heap allocation occurs — stack memory is typically a statically declared array. Use the static variant in safety-critical or memory-constrained systems where heap fragmentation is unacceptable, or when you need guaranteed allocation at compile time. The trade-off is that you must size the stack array manually and waste memory if over-estimated.

In FreeRTOS, what is the difference between `xTaskCreate` and `xTaskCreateStatic`? When would you use the static variant?

SA7

A moving average computes the arithmetic mean of the last N samples. A short window (N=4) responds quickly to genuine signal changes but provides little noise suppression — high-frequency noise still appears in the output. A long window (N=32) smooths out noise effectively but introduces latency: a step change in the physical signal takes up to 32 samples to fully propagate to the output, potentially delaying threshold detection. Choose N based on the expected rate of physical change relative to the measurement noise power — faster dynamics require smaller N.

What is a moving-average filter, and what is the trade-off between a short window (e.g., N=4) and a long window (e.g., N=32) for smoothing sensor readings?

SA8 What is the difference between `#pragma once` and a traditional `#ifndef` include guard in C? Are there any portability concerns?	A traditional include guard uses `#ifndef MYFILE_H / #define MYFILE_H / ... / #endif` to prevent a header from being included twice. `#pragma once` achieves the same effect with a single directive and is supported by all major compilers (GCC, Clang, MSVC). The portability concern is that `#pragma once` is not mandated by the C/C++ standard — it is a widely-supported extension. In practice it is safe for almost all modern embedded toolchains, but legacy or exotic compilers may not support it. `#pragma once` also has a subtle difference: it is file-identity-based (inode or path), while include guards are name-based; this matters in edge cases involving hard links or unusual build systems.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
DA1 Calculate the approximate time-on-air for a LoRa packet with the following parameters: SF12, BW 125 kHz, CR 4/5, 20-byte payload, explicit header. Show your reasoning. Then explain how the EU 868 MHz 1% duty-cycle limit constrains the minimum inter-message interval, and what this means for the design of a sensor that must report every 5 minutes.	The symbol duration for LoRa is `Ts = 2^SF / BW`. At SF12, BW 125 kHz: `Ts = 4096 / 125000 ≈ 32.77 ms`. A preamble of 8 symbols takes `8 × 32.77 ≈ 262 ms`. The number of payload symbols is computed by the LoRa formula: with CR=4/5, 20-byte payload, explicit header enabled, the payload symbol count is approximately 40 symbols (using the standard ceiling formula from the Semtech datasheet). That gives `40 × 32.77 ≈ 1311 ms` of payload ToA. Total ToA ≈ 1575 ms ≈ 1.6 seconds. The EU 868 MHz 1% duty-cycle rule means: after a 1.6 s transmission, the device must remain silent for at least `1.6 / 0.01 - 1.6 = 158.4 s ≈ 2.6 minutes`. Since 2.6 minutes < 5 minutes, the sensor can legally report every 5 minutes — the duty cycle constraint is satisfied with margin. However, energy per message is high at SF12: at a typical TX current of 120 mA and 3.3 V, 1.6 s ≈ 0.176 mJ. A device targeting years of battery life on AA cells should consider whether SF7 or SF8 would provide sufficient link budget; at SF7 ToA drops to ~50 ms, reducing energy per message by ~30× and freeing duty-cycle headroom for the same 5-minute interval.
DA2 Explain Elixir's "let it crash" philosophy. When is it the right approach, and when is it wrong to rely on it? Cover: what supervisors do, what kinds of errors are recoverable via restart, what kinds require explicit error handling, and how this differs from defensive programming in C.	"Let it crash" means that instead of writing defensive code to handle every possible error inline, you allow processes to crash on unexpected conditions and rely on a supervisor to restart them in a known-good state. This works because Elixir processes are isolated: a crash affects only that process and its dependents, not the whole system. The supervisor's restart strategy (`:one_for_one`, `:one_for_all`, `:rest_for_one`) determines which sibling processes are also restarted. When it is right: transient errors that are self-healing on retry — a network timeout, a temporary sensor glitch, a database connection drop. Restarting the process resets all state cleanly without leaving inconsistent data structures. When it is wrong: errors that will recur immediately on restart (a bad configuration, an unreachable dependency at startup) will cause rapid crash-restart loops (hitting the supervisor's max-restart threshold), eventually taking down the supervisor tree. These must be handled explicitly — validate configuration before starting a process, use `:ignore` or `{:error, reason}` returns from `init/1` instead of crashing.

	This differs fundamentally from C: in C there is no isolation — a null pointer dereference corrupts the process address space. Defensive `if (ptr == NULL) return -1;` checks are the only option. In Elixir the VM enforces isolation, making "crash and restart" a viable strategy that in practice produces simpler, shorter code with cleaner separation between the happy path and error recovery.
DA3 Explain how stream processing can be structured in an Elixir IoT backend. Compare `Enum`, `Stream`, `Flow`, and `GenStage` — what each one is, when you would use it, and how they map to the stages of an IoT data pipeline (ingestion → filtering → aggregation → alerting).	`Enum` is eager: functions like `Enum.map/2` and `Enum.filter/2` process the entire collection in memory and return a new list. Use it for small, finite datasets where simplicity matters — transforming a batch of sensor readings from a database query. `Stream` is lazy: operations are composed into a pipeline that pulls one element at a time, only executing when a terminal function (`Enum.to_list`, `Stream.run`) forces evaluation. Use it for large or potentially infinite sequences — reading a CSV file line by line, or processing a GenStage producer's output — to avoid loading the entire dataset into memory. `Flow` (from the `flow` library) adds parallelism: it partitions a `Stream` across multiple worker processes, processes elements concurrently, and merges results. Use it for CPU-bound batch processing — feature extraction, data normalisation — where the pipeline can be parallelised safely. `GenStage` is the foundation for back-pressure-aware producer/consumer pipelines. A producer emits events; consumers demand them using explicit demand signals. If a consumer is slow, demand backs up and the producer slows down, preventing buffer overflow. Use it for the full IoT ingestion pipeline: a producer reads from an MQTT broker or TCP socket, intermediate stages filter and decode, a consumer stage writes to a time-series database. `Flow` is built on top of `GenStage` and hides the demand protocol for simpler use cases. In an IoT pipeline: ingestion (MQTT subscriber → GenStage producer) → decoding/filtering (`Flow` parallel stage) → aggregation (`Stream` windowing with `Stream.chunk_every`) → alerting (`GenStage` consumer that calls a notification API when a threshold is crossed).

Section 5 — Design Question

QUESTION	MODEL ANSWER
18 A city council wants to deploy a smart parking system across a **surface lot of 200 spaces** (400 m × 100 m). Each space has a sensor that detects whether it is occupied. The system should: • **Show real-time occupancy** on a public display at the lot entrance (updated every 30 seconds). • **Allow drivers** to query availability via a mobile app. • **Charge parked vehicles** automatically: the system must know when each vehicle arrived and left in order to bill correctly. • **Alert maintenance staff** when a sensor appears to have failed (no state change for 7 days). Each sensor node is battery-powered (2 × AA). There is no wired infrastructure.	—

Section 3 — Short Answer

QUESTION	MODEL ANSWER
18.1 Compare two sensor technologies for detecting vehicle presence (e.g., magnetic vs. ultrasonic). Choose one and justify the choice considering power budget, installation, and false-positive rate.	Inductive loop / magnetometer sensors detect the distortion of Earth's magnetic field caused by a vehicle's steel body. They are passive once deployed, consume very little power (µA range when polling slowly), are immune to rain, snow, and ambient light, and have a very low false-positive rate because only ferromagnetic masses trigger them. The main drawback is that installation requires cutting the loop into the asphalt (inductive loop) or burying the sensor, which is disruptive. A magnetometer surface-mounted in a housing is easier but sensitive to calibration drift. Ultrasonic sensors measure round-trip time of a 40 kHz pulse to detect distance to the ground (absent vehicle) vs. a vehicle roof. They work regardless of vehicle material but consume more power (mA during active pulse), are affected by temperature (speed of sound varies), and can give false readings from debris, birds, or heavy rain. They are better suited for indoor garages with stable temperature. **Choice: magnetometer.** For an outdoor surface lot, the magnetometer wins on power budget (years of AA life vs months for ultrasonic), robustness to weather, and lower false-positive rate. Installation in a small surface-mounted puck is acceptable and avoids road cutting.

Section 4 — Detailed Answer

QUESTION	MODEL ANSWER
18.2 Design the data pipeline from sensor to mobile app and billing backend. What radio technology do you use from sensor to gateway? Where do you run the occupancy state machine (edge/fog/cloud)? How do you ensure billing accuracy if a gateway reboots mid-session?	**Sensor to gateway:** Use LoRaWAN. The lot spans 400 m × 100 m; a single LoRaWAN gateway covers the entire area with margin. Magnetometer sensors change state infrequently (when a car arrives or departs), so duty-cycle constraints are not a problem — occupancy events are short packets (~10 bytes) well within the 1% limit. Each node runs on 2 × AA for 2–5 years at SF9/SF10. **Occupancy state machine:** Run on a fog-tier server co-located with the gateway (e.g., a Raspberry Pi or small x86 box on-site). The state machine holds a `{space_id → {occupied | free, arrived_at}}` map. When a "vehicle detected" event arrives, it records the arrival timestamp; when a "vehicle gone" event arrives, it closes the session. Running this at the fog tier means the state survives internet outages — occupancy counts continue to update, the entrance display (local LAN) continues to refresh, and session records accumulate locally. **Billing accuracy through gateway reboots:** Session records (space_id, arrival_time, departure_time) are written to a local SQLite database with a `synced` flag. On reboot, the fog node re-reads open sessions from the DB and resumes. Once the internet reconnects, it bulk-syncs unsynced records to the cloud billing backend. LoRaWAN Class A devices retransmit unacknowledged uplinks, so a brief gateway reboot does not lose events — they are queued in the device's retransmit buffer. For billing disputes, a tamper-evident log of raw LoRaWAN payloads (timestamped by the gateway hardware clock, synchronised via NTP) serves as the audit trail.

	Mobile app and billing: The fog node exposes a REST/WebSocket API consumed by the entrance display and the mobile app (via a cloud reverse-proxy that tunnels to the fog API). The billing backend receives completed session records via HTTPS from the fog node.

Section 3 — Short Answer

QUESTION	MODEL ANSWER
18.3 A sensor has reported no state change for 7 days. Describe how you distinguish a genuinely empty space from a failed sensor, and what automated action the system should take.	A sensor that genuinely covers an empty space will report no state change. To distinguish it from a faulty sensor, the system can: (1) check whether the space has been occupied at all in the past 7 days, compared to adjacent spaces that show normal occupancy cycling — a healthy empty space is plausible at 3 AM but implausible if every adjacent space shows daily turnover; (2) send a LoRaWAN downlink "ping" to the node requesting a status response including battery voltage and sensor self-test result — if the node responds with healthy status, the space is genuinely unoccupied; if there is no response after 3 retries across 24 hours, the node is presumed dead. Automated action: raise a maintenance alert to staff with the space ID and last-seen timestamp, suppress the space from the occupancy count so billing logic does not incorrectly assume it is free, and schedule a physical inspection. The system should not automatically mark the space as "occupied" (which would block billing), but should mark it as "unknown" with reduced confidence.